arXiv:2605.27788v1 [cs.LG] 27 May 2026

Knowing When to Ask: Segment-Level Credit Assignment for LLM Tool Use

Abhijit Kumar* **Zoey Wu** **Mohit Suley**
Microsoft AI
Redmond, WA
{abhijitkumar, guwu, mosuley}@microsoft.com

Abstract

Humans know when to reach for help e.g. 347×28 warrants a calculator while $2 + 2$ does not. Language models, by default, do not. Prompt-based approaches can instruct a model when to invoke tools, but this external scaffolding does not teach the model to recognize the boundary of its own knowledge. Reinforcement learning approaches that assign a single outcome reward to the whole trajectory fare no better: trajectory-level credit cannot isolate which tool call in a successful episode actually helped, nor penalize unnecessary calls. We propose **CARL** (**C**ompetence-**A**ware **R**einforcement **L**earning), which trains a critic on the model's own rollouts to learn where the model's parametric knowledge suffices and where it needs external help. By decomposing each rollout at natural tool-use boundaries (e.g., code fence delimiters and context block transitions), CARL assigns independent credit to each segment from a single binary outcome, without external judges or step-level annotations, addressing the credit-assignment limitations of trajectory-level methods. As a result, erroneous tool calls, incorrect extractions, and unnecessary calls each receive appropriately signed advantages. We show that the trained critic captures the model's domain competence: it separates parametrically solvable from tool-dependent questions with AUC 0.93 at 7B. On five benchmarks spanning arithmetic, multi-hop factual QA, and numerical reasoning over financial tables, CARL improves exact-match accuracy by 6.7 points at 7B and 9.7 points at 3B over the best RL baseline (Search-R1 PPO at 7B, Search-R1 GRPO at 3B), with the largest gain (+8.3 EM at 7B, +9.0 EM at 3B) on Musique, the most compositional multi-hop benchmark. Compared to the best RL baseline (Search-R1 PPO), the model issues 53% fewer tool calls on questions answerable from parametric knowledge while remaining $\sim$10 EM points more accurate on those same questions, an emergent consequence of the critic learning where this model's competence ends. Gains are largest at small scale: the 3B model improvement is $1.4\times$ the 7B improvement, suggesting that knowing when to ask for help disproportionately benefits models with smaller parametric memory.

1 Introduction

Out of the box, large language models do not come equipped with the machinery that makes their outputs reliable. They lack reliable access to current information, arithmetic precision, and a means of verifying their own claims. Prompt-based techniques such as retrieval-augmented generation, ReAct, and tool-augmented prompting typically bolt a calculator and a search box onto the model at inference time, yet the model still invokes tools without knowing whether the task actually requires them. Humans do not behave this way. We do not need to be told that 347×28 warrants a calculator

*Corresponding author.

Preprint. Under review.

while $2 + 2$ does not, or that the capital of an unfamiliar country requires a search while the capital of our own does not. We recognize the edge of our own knowledge and reach for help precisely when we need it. Our goal is to train language models to do the same, not by being instructed when to use tools, but by learning it.

Recent RL approaches train LLMs on tool-use trajectories with a single outcome reward (6; 7; 9; 21; 22), creating two coupled problems: credit assignment (a good search and a bad search in the same successful trajectory are reinforced identically) and tool-use selectivity (model lacks awareness of its own competence boundary, so unnecessary tool calls still receive positive reinforcement). Prior attempts enrich the reward or score steps with an external judge (§2), but either retain a single trajectory-level scalar or rely on a judge whose knowledge may diverge from the student's.

Our starting observation is that tool-use trajectories have something pure reasoning trajectories lack: *structurally observable boundaries* (§3.1, Figure 1). At each boundary the context changes observably making a central RL problem tractable: identifying meaningful decision boundaries in generation, a challenge discussed at length in prior work (15; 13; 2). A critic that estimates, at each boundary, the probability that the current context will lead to a correct answer can be trained from binary outcome reward alone without requiring step-level annotations or ambiguous step-boundary heuristics. Given such a critic, the advantage of a segment is simply how much it changed the expected probability of success e.g. a good search raises it, a bad search lowers it, an unnecessary search on an easy question leaves it roughly unchanged. Because the critic is trained on *this model's own rollouts*, it learns a competence-aware value function, one that reflects what this model can do.

We instantiate this as CARL (**C**ompetence-**A**ware **R**einforcement **L**earning). Our contributions:

1. **A value function that encodes the model's own competence boundary.** From binary outcome reward alone, $V(s_0)$ learns to separate parametrically solvable from tool-dependent questions (AUC 0.93 at 7B, 0.85 at 3B), producing 53% fewer unnecessary tool calls than the best RL baseline.
2. **Per-segment credit from a single outcome.** We derive per-segment advantages from the Semi-Markov Decision Process (SMDP) (12) Bellman equation at structurally observable tool-use boundaries. A good search paired with a bad extraction produces opposing signs within the same trajectory. Trajectory-level methods cannot make this distinction.

On five benchmarks, CARL improves over Search-R1 PPO, the strongest RL baseline, on the three search-based datasets, with the largest gains on multi-hop out-of-distribution tasks and up to 36% fewer tokens. Gains are largest at small scale ($1.4\times$ at 3B vs. 7B).

2 Related Work

Three lines of work are most relevant. Prompt-based methods (RAG, ReAct) attach tools at inference time but cannot train selectivity. Trajectory-level RL methods train on outcome reward but cannot assign differential credit within a trajectory. Step-level credit methods attempt finer credit but rely on external judges whose knowledge may diverge from the student's. We position CARL against all three, with trajectory-level RL as our direct experimental baseline.

Trajectory-level RL for tool use. Search-R1 (6), ToRL (7), ReSearch (21), and R3-RAG (22) train LLMs on tool-use trajectories with a single outcome reward. Because the reward is shared across all segments, the contribution of any single tool call is hard to isolate, and unnecessary tool calls on easy questions still receive positive reinforcement when the episode succeeds. ToolRL (9) enriches the reward with tool-name and argument-quality components but applies the result as a single trajectory-level scalar, so it refines what the reward evaluates without changing which segment each signal reaches. StepTool (27) scores each step with GPT-4 during PPO, but the external judge can only score calls that were made, not penalize calls that should not have been, and its knowledge may diverge from the student's. The two approaches are orthogonal: a richer ToolRL-style reward could in principle be combined with CARL's segment-level credit assignment.

Options, SMDPs, and credit assignment. CARL instantiates the Options framework (12): each segment is an option, termination fires at a structural delimiter, and the SMDP Bellman equation

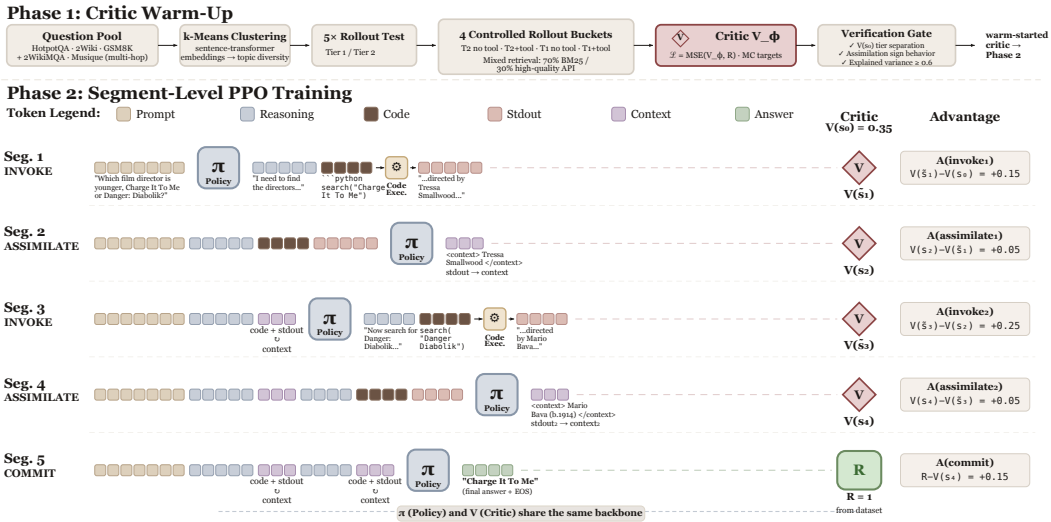


Figure 1: **Rollout pipeline in CARL.** Each tool-use trajectory decomposes into three segment types (**invoke**, **assimilate**, **commit**) with structurally observable boundaries. After invoke, the rollout loop executes the code and captures stdout; after assimilate, it replaces the raw stdout with the model's `<context>` block. The critic V_ϕ evaluates the context at each boundary; the advantage of each segment is the change in V_ϕ across it, with the terminal reward R replacing the final V_ϕ at EOS.

gives the per-segment advantage (§3.2). SPO (14) independently arrives at segment-level credit but uses arbitrary cutpoints and critic-free MC estimation on single-turn math; CARL's boundaries are SMDP environment transitions and its critic encodes a cross-question competence boundary (Appendix L). Recent work applies Options vocabulary to LLM agents (ARC (3), Agent-as-Tool (4)) but trains with trajectory-level GRPO; CARL derives per-option advantages from a value function. HICRA (5) addresses credit assignment differently, heuristically weighting optimization toward attention-identified planning tokens. Among credit-assignment methods, EGCA (2) localizes credit in code generation via execution-trace comparison but requires reference solutions; PAVs (13) share our $V(s_{t+1}) - V(s_t)$ form but require step-level annotation; VinePPO (15) found token-level value networks unreliable, a problem our segment-level setting avoids because boundaries number in single digits and each corresponds to a large context change. Earlier return-decomposition work (RUDDER (1)) shares the potential-based form of our advantages but operates at the token level. Extended per-paper comparisons are in Appendix L.

3 Methodology

CARL has three components: a decomposition of tool-use trajectories into segments with structurally observable boundaries (§3.1), a value-function-based advantage derived from the SMDP Bellman equation (§3.2), and a critic trained on outcome reward with curated warm-up data (§3.3). These plug into standard PPO (11) (§3.4).

3.1 Segment Decomposition and Tool Interface

A tool-use trajectory consists of a sequence of segments, each an option in the sense of Sutton et al. (12). A trajectory with K tool calls has $2K + 1$ segments; a trajectory with no tool calls is a single commit segment. The tool interface is Python code blocks: the model writes arithmetic, calls a predefined `search()` function, or implements custom logic. Figure 1 shows the rollout control flow.

The *invoke* segment generates reasoning followed by a code block (terminated at the closing code fence); the rollout loop executes the code and appends stdout o_k^{raw}. The *assimilate* segment reads tool output and writes a `<context>` block distilling the relevant information (terminated at `</context>`); the rollout loop replaces o_k^{raw} with the extraction e_{k+1}. The *commit* segment produces the final

answer (terminated at EOS). Because raw tool output is replaced before the next generation begins, each segment's target contains only model-produced tokens and gradient masking is unnecessary (formalized in Appendix B).

Reward. A single terminal reward $R = \text{EM}(\text{prediction}, \text{gold}) \in \{0, 1\}$ after standard normalization. No intermediate rewards, execution penalties, or format rewards.

3.2 Segment-Level Advantage

Given only a terminal reward $R \in \{0, 1\}$, how do we assign credit per segment? From the SMDP Bellman equation (Appendix A), under terminal-only reward and finite horizon ($\gamma = 1$), the per-segment advantage reduces to:

$$A(\text{seg}_k) = \begin{cases} V_\phi(s_{k+1}) - V_\phi(s_k), & k < N \quad \text{(intermediate segment)}, \\ R - V_\phi(s_N), & k = N \quad \text{(final segment)}. \end{cases} \tag{1}$$

For a single tool-interaction trajectory with segments (invoke, assimilate, commit), this yields three independent advantages from one binary outcome. Summing per-segment advantages telescopes to $R - V_\phi(s_0)$, preserving the same total credit as trajectory-level PPO (Appendix A).

Per-segment advantages in a tool interaction. Each invoke–assimilate pair passes through two distinct states: a *transient* state $\tilde{s}_{k+1}$, containing the raw tool output immediately after invoke, and the next *persistent* state s_{k+2}, in which the assimilate segment has replaced raw output with its `<context>` block (formalized in Appendix B). The critic V_ϕ evaluates at both, yielding separate advantages for query quality and extraction quality. For a single tool call starting at boundary k, Equation 1 produces the following invoke–assimilate–commit triple:

$$\begin{aligned} A(\text{invoke}) &= V_\phi(\tilde{s}_{k+1}) - V_\phi(s_k) && \text{(query quality)}, \\ A(\text{assimilate}) &= V_\phi(s_{k+2}) - V_\phi(\tilde{s}_{k+1}) && \text{(extraction quality)}, \\ A(\text{commit}) &= R - V_\phi(s_{k+2}) && \text{(answer commitment)}. \end{aligned} \tag{2}$$

All tokens within a segment share their segment's advantage, enabling invoke and assimilate to receive opposing credit within the same trajectory.

3.3 The Critic

Architecture. A two-layer MLP value head attached to the shared LLM backbone produces $V_\phi(s) \in [0, 1]$ at each segment boundary. Critic gradients update both the head and backbone, with the backbone learning rate $10\times$ smaller to preserve policy representations (architecture details in Appendix D).

Training target. The critic is trained with Monte Carlo targets: MSE between $V_\phi(s)$ and the terminal reward R, averaged across boundary states. MC targets are preferred because they are unbiased and their variance is bounded by the binary reward.

Warm-up. Critic warm-up before PPO has been demonstrated to help in prior RLHF work (23) and is essential here: without it, early random critic gradients corrupt the shared backbone, and the policy must unlearn them. The warm-up set is built from base-model rollouts labeled by final correctness, with no human annotation or external judge. Questions are classified as Tier 1 (beyond parametric competence) or Tier 2 (within it) via multi-rollout consistency, and each is rolled out under forced-tool and no-tool prompts. This produces four outcome buckets that teach V_ϕ to separate what the model knows from what it does not (details in Appendix C). This warm-up is also applied to our Search-R1 PPO baseline so that both methods start from the same critic-quality baseline; even with matched warm-up, segment-level credit retains a margin that grows with critic calibration (Appendix H). The cold-start ablation (no warm-up) trails our full method by $\sim$9 EM on HotpotQA at 7B (Appendix H). Figure 2(a) shows the resulting calibration curves: curated warm-up tracks the diagonal (ECE = 0.043), random-data compresses the slope (ECE = 0.137), and cold-start is near-flat (ECE = 0.238).

Table 1: Dataset characteristics. Tier 1 is the fraction of evaluation questions the base model answers incorrectly without tool access.

Dataset	Tool	Hops	Tier 1 (3B)	Tier 1 (7B)	In training?
GSM8K	Python	1	8%	3%	Yes
HotpotQA	BM25 search	2	84%	76%	Yes
2WikiMQA	BM25 search	2	68%	57%	Yes
FinQA	Python	2–3	95%	91%	No
Musique	BM25 search	2–4	96%	93%	No

3.4 Training Objective

With segments defined (§3.1), advantages derived (§3.2), and the critic trained (§3.3), the training objective is standard PPO applied at the segment level. For each segment k, the PPO clipped surrogate objective is

$$\mathcal{L}_k^{\text{CLIP}} = \frac{1}{|g_k|} \sum_{t \in g_k} \min\Big(r_t(\theta)\, A(\text{seg}_k),\ \text{clip}\big(r_t(\theta),\, 1-\epsilon,\, 1+\epsilon\big)\, A(\text{seg}_k) \Big), \tag{3}$$

where g_k is the set of model-generated tokens in segment k and $r_t(\theta) = \pi_\theta(a_t \mid c_t)/\pi_{\theta_{\text{old}}}(a_t \mid c_t)$ is the per-token importance ratio. Each segment receives its own advantage scalar, applied uniformly to all tokens within it. This enables invoke and assimilate to receive opposing credit when query quality and extraction quality disagree.

The critic is trained with the MC loss introduced in §3.3, formalized as $\mathcal{L}^{\text{critic}} = \frac{1}{N+1} \sum_{k=0}^{N} (V_\phi(s_k) - R)^2$. The total trajectory loss combines the two: $\mathcal{L} = -\sum_{k=0}^{N} \mathcal{L}_k^{\text{CLIP}} + c_v \mathcal{L}^{\text{critic}}$, with the advantage treated as constant during policy-gradient computation.

4 Experiments

We evaluated CARL on five benchmarks spanning two tool types, two model scales, and a range of credit-assignment complexity. Our experiments answer three questions: does CARL outperform trajectory-level baselines (§4.3), does the critic actually learn the competence boundary we claim (§4.4), and what bounds the remaining gains (§4.5)? We also verify that GSM8K performance is preserved (97% Tier 2 at 7B), that baseline generative capabilities remain intact when tools are removed (Appendix K, Table 15), and that the model degrades gracefully when tools return useless output (Appendix K, Table 16).

4.1 Datasets

We selected five benchmarks that jointly vary two axes: the typical number of tool calls per question, and the fraction of questions the base model can answer from parametric knowledge alone. A *hop* refers to one invoke–assimilate pair requiring an external tool call. We classify each question as **Tier 2** (within parametric competence) if the base model answers it correctly in at least one of five independent no-tool rollouts, and **Tier 1** (beyond parametric competence) if all five rollouts produce incorrect answers. Table 1 summarizes the five benchmarks.

GSM8K (16) is our pure selectivity test: almost entirely Tier 2, the question is whether the model learns to skip calling a tool. HotpotQA (17) sits in the middle of the tool-necessity spectrum. Among the three search-based benchmarks, 2WikiMQA (18) has the highest Tier 2 rate among search benchmarks (43% at 7B), making it the strongest test of learning to skip searches the model does not need (Table 1). FinQA (19) is held out from training and tests generalization to heterogeneous tool routing (search followed by computation). Musique (20) is our hardest benchmark: compositional 3–4 hop questions, also held out.

Training data. We train on the merged training splits of the three in-training datasets (HotpotQA, 2WikiMQA, GSM8K), totaling approximately 170K questions, matching the data scale of Search-R1 (6). FinQA and Musique are reserved entirely for out-of-distribution evaluation. For search benchmarks, we use BM25 retrieval over per-dataset Wikipedia passages with top-$k = 3$.

Evaluation. We evaluate on the full development sets, following the protocol of Search-R1 (6) and ReSearch (21). Exact Match is computed after normalization. For multi-word entity answers, we also accept predictions whose normalized form contains the normalized gold as a substring.

4.2 Methods

We compare four trained methods and one prompting baseline. Each of the three RL methods occupies a distinct point on the credit-assignment granularity spectrum: Search-R1 GRPO applies a single trajectory-level scalar to all tokens, Search-R1 PPO uses a token-level value function, and CARL applies a segment-level value function at invoke/assimilate/commit boundaries. Reading the main table in order traces coarse $\rightarrow$ fine $\rightarrow$ boundary-aligned credit.

CoT + Tools. A ReAct-style agent loop (26) over Qwen2.5-7B-Instruct with access to the same retrieval corpus as the trained methods (BM25, inline tables, Python); no fine-tuning. We run *always-use* (tools on every question) and *optional-use* (model discretion) variants as references for retrieval quality and prompting-only selectivity respectively.

SFT (rejection-sampled). We supervised-fine-tune Qwen2.5-Instruct on its own correct (EM=1) tool-use trajectories, providing a supervised-learning baseline without RL credit assignment.

Search-R1 (GRPO and PPO). We trained both variants in the open-source codebase (6) on our training data with matched retrieval infrastructure, carefully tuning hyperparameters to give these baselines every advantage. For Search-R1 PPO, we additionally apply the same curated critic warm-up as CARL so that both methods start from the same critic quality; we report results with and without warm-up in Appendix H, where even the strongest Search-R1 PPO configuration exceeds published numbers yet CARL retains a margin that grows with calibration.

CARL (ours). Segment decomposition (§3.1) with segment-level PPO (§3.4). Full hyperparameters are in Appendix D; retrieval setup, compute, and reproducibility details are in Appendix O.

Methods using richer supervision (e.g. StepTool's GPT-4 judgments) operate under a different assumption and are not directly comparable; we discuss them in §2 and Appendix L.

4.3 Main Results

Table 2 reports per-dataset accuracy and average tokens per episode across all methods and both scales. Search-R1 is a search-only framework and does not support non-search tools (e.g., calculator for GSM8K/FinQA); we therefore compare against Search-R1 PPO and GRPO on the three search-based datasets (HotpotQA, 2WikiMQA, Musique). The table supports three claims.

Credit-assignment granularity matters most at the boundary level. [2] At 7B, moving from trajectory-level credit (Search-R1 GRPO) to token-level credit (Search-R1 PPO) adds +4.0 average EM. Moving from token-level to segment-level (CARL) adds a further +6.7. The gain from aligning the critic's boundaries with semantic transitions is larger than the gain from making credit finer-grained at the token level: segment-level credit operates at positions where the state actually changes, while token-level credit operates mostly at positions where it does not.

Gains concentrate on multi-hop questions. Per-dataset CARL gains over Search-R1 PPO at 7B are +5.1 on HotpotQA, +6.6 on 2WikiMQA, and +8.3 on Musique. The Musique gain, on a 3–4 hop fully held-out benchmark, is the largest. The pattern repeats and amplifies at 3B: +10.2, +12.6, and +9.0 respectively, with the average 3B gain on these three search datasets (+10.6) running $1.6\times$ the corresponding 7B gain (+6.7) (Appendix J). Smaller models benefit disproportionately, consistent with the abstract's claim that knowing when to ask matters most when parametric memory is small (Appendix J decomposes this scaling asymmetry into three contributing mechanisms). SFT lifts in-domain HotpotQA and 2WikiMQA over the prompting ceiling (+2–6 EM at both scales) but does

[2] Evaluation subsets vary by analysis. Table 2 evaluates all methods on the complete dev set of each benchmark ($\sim$2,500 questions total). Table 3 restricts to Tier 2 (parametrically solvable) questions from the three search benchmarks ($\sim$370 questions) to isolate selectivity behavior. Figure 2(b) tracks held-out accuracy over training on four benchmarks (excluding GSM8K, which is calculator-only). Figure 2(c) plots Tier 2 tool-use rate across all five benchmarks.

Table 2: **Main results.** Accuracy or EM (↑) and generation length in tokens (↓). †Training data; dashes indicate Search-R1 is not applicable. CARL and Search-R1 PPO report mean±std over 3 seeds; tokens are means. Prompting baselines, SFT, and GRPO are single runs.

		GSM8K		HotpotQA†		2Wiki†		FinQA		Musique		Avg_5 / Avg_3	
Policy	Method	Acc	Tok	EM	Tok	EM	Tok	Acc	Tok	EM	Tok	Sc	Tok
3B-Inst.	CoT+Tools (always)	86.3	823	22.2	1943	19.4	2028	38.4	1962	4.6	2781	34.2 / 15.4	1907 / 2251
	CoT+Tools (selective)	86.7	391	19.1	1685	21.2	1732	28.2	2096	5.3	2792	32.1 / 15.2	1739 / 2070
	SFT (rej. sampled)	81.8	412	24.6	1818	25.1	1791	25.7	2014	4.8	2867	32.4 / 18.2	1780 / 2159
	Search-R1 GRPO	–	–	34.3	1921	34.3	1960	–	–	8.6	2778	– / 25.7	– / 2220
	Search-R1 PPO	–	–	30.2±0.5	1809	32.1±0.6	1861	–	–	12.1±0.4	2755	– / 24.8	– / 2142
	CARL (ours)	**87.2**±0.3	**384**	**40.4**±0.2	**1324**	**44.7**±0.3	**1394**	**46.6**±0.4	**1737**	**21.1**±0.2	**2546**	**48.0 / 35.4**	**1477 / 1755**
	Δ *vs. 2nd*	+0.5	↓7	+6.1	↓361	+10.4	↓338	+8.2	↓225	+9.0	↓209	+13.8 / 9.7	↓262 / 315
7B-Inst.	CoT+Tools (always)	91.8	781	30.8	1862	26.6	1938	56.2	1623	11.4	2674	43.4 / 22.9	1776 / 2158
	CoT+Tools (selective)	91.4	367	31.1	1621	27.1	1641	55.7	1785	9.8	2708	43.0 / 22.7	1624 / 1990
	SFT (rej. sampled)	89.4	389	33.7	1712	32.6	1683	51.3	1864	8.9	2756	43.2 / 25.1	1681 / 2050
	Search-R1 GRPO	–	–	39.1	1779	36.8	1827	–	–	18.5	2579	– / 31.5	– / 2062
	Search-R1 PPO	–	–	42.8±0.6	1837	41.7±0.7	1775	–	–	22.1±0.6	2682	– / 35.5	– / 2098
	CARL (ours)	**91.5**±0.2	**341**	**47.9**±0.3	**1193**	**48.3**±0.4	**1139**	**59.3**±0.4	**1389**	**30.4**±0.4	**2511**	**55.5 / 42.2**	**1315 / 1614**
	Δ *vs. 2nd*	−0.3	↓26	+5.1	↓428	+6.6	↓502	+3.1	↓234	+8.3	↓68	+12.1 / 6.7	↓309 / 376

Paired Δ *(CARL$_i$ − SR1-PPO$_i$, same seed) — 7B: HotpotQA* +5.1±0.3*, 2Wiki* +6.6±0.5*, Musique* +8.3±0.4*; 3B:* +10.2±0.4*,* +12.6±0.4*,* +9.0±0.3*.*

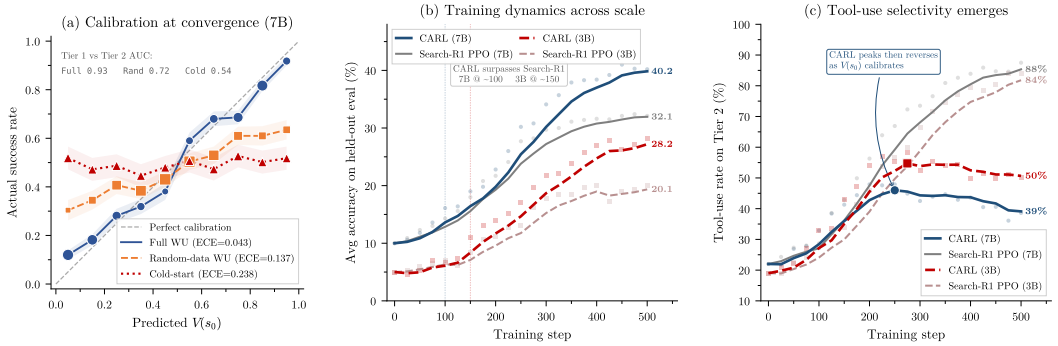


Figure 2: $V(s_0)$ calibration drives faster training and emergent selectivity. **(a)** Calibration of $V(s_0)$ under three warm-up regimes (Appendix H). **(b)** Held-out accuracy across 500 PPO steps (4-dataset average). CARL surpasses Search-R1 PPO at step ∼100 (7B) / ∼150 (3B). **(c)** Tier 2 tool-use rate. CARL peaks then settles (∼46% at 7B); Search-R1 PPO climbs monotonically toward 84–88%.

not transfer: it regresses on FinQA and is essentially neutral on Musique, suggesting that imitation captures successful patterns but not selective tool use.

Tool-use efficiency improves substantially. At 7B on the three PPO-comparable datasets, CARL averages 1,614 tokens per episode versus 2,098 for Search-R1 PPO, a 23% reduction concentrated on the search tasks where calibrated $V(s_0)$ lets the model skip retrieval on Tier 2 questions[3] (35–36% fewer tokens on HotpotQA and 2WikiMQA). Musique tokens remain high (−6%) because its questions are inherently Tier 1 and require retrieval at every hop. Per-episode tool-call counts (Appendix E) confirm that the reduction is driven by selective invocation.

4.4 How and Why CARL Works

We next show that the critic learns what the model knows, and that this drives both faster training (Figure 2b) and selective tool use (Figure 2c, Table 3).

A calibrated competence boundary. $V(s_0)$ classifies Tier 1 from Tier 2 questions with AUC 0.93 at 7B and 0.85 at 3B (Figure 4; calibration shape in Figure 2(a)). Calibration matters as well as ranking: random-data warm-up reaches AUC 0.72 but ECE 0.137, producing advantages that carry

[3]Tier 2 = answerable from parametric knowledge; Tier 1 = requires tool use to answer correctly. Definition: §4.1

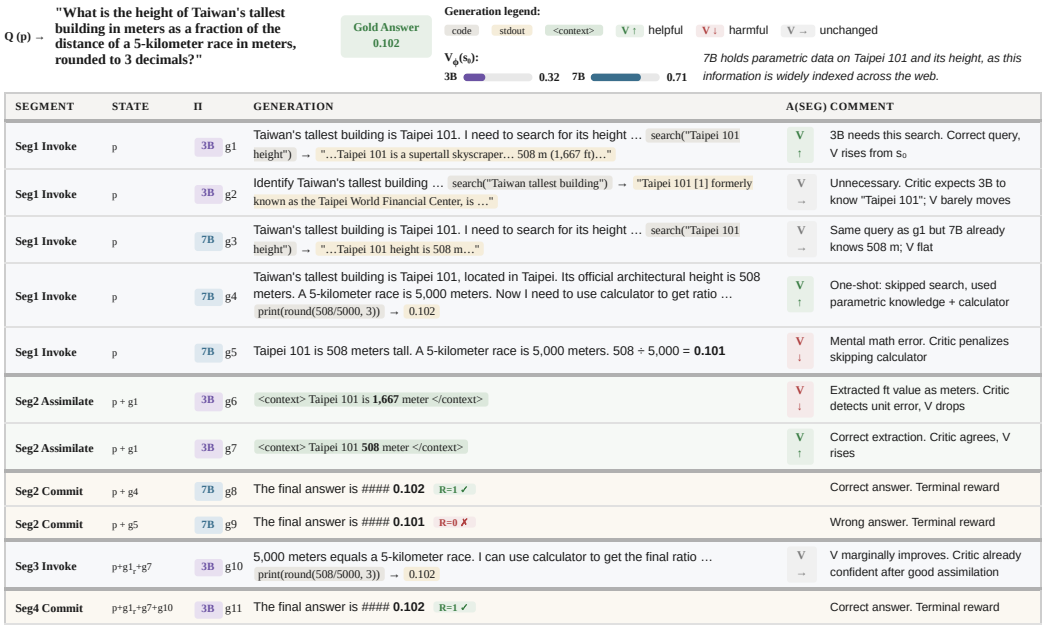


Figure 3: **Qualitative analysis of the CARL critic on a compositional query.** The same prompt elicits distinct critic behaviour at each scale ($V(s_0)$: 3B = 0.32, 7B = 0.71). **(i)** A search raises V at 3B but not 7B, which already knows the answer. **(ii)** Faithful extraction raises V; a unit-error extraction lowers it, showing opposing signs within one trajectory. **(iii)** At 7B, calculator use earns $V\uparrow$ while mental math earns $V\downarrow$. All advantages arise from binary reward alone.

Table 3: Tool-use selectivity at convergence (Qwen2.5-7B), evaluated over Tier 2 questions from HotpotQA, 2WikiMQA, and Musique (∼370 questions total). These are the three datasets where Search-R1 was trained in its public release, so all comparisons are apples-to-apples. Tier 1 tool rate confirms CARL still invokes tools when needed. Tier 2 tool rate measures unnecessary tool calls. Tier 2 EM measures accuracy on these questions, regardless of whether the method invoked tools.

Method	Tier 1 tool rate	Tier 2 tool rate	Tier 2 EM
CoT+Tools (always)	100.0%	100.0%	72.4
Search-R1 GRPO	96.2%	87.5%	75.1
Search-R1 PPO	95.4%	82.6%	77.2
CARL (ours)	**97.8%**	**38.6%**	**87.0**

the right sign more often than the right magnitude. The critic also signs intermediate steps correctly: V rises after successful extraction 79.4% of the time and drops when assimilation loses a surfaced answer (72.1%), a within-trajectory sign divergence that trajectory-level methods cannot produce (per-segment sign statistics in Appendix I.1, Table 12).

Calibration drives faster training. Once explained variance crosses 0.60 (around step ∼100 at 7B, ∼150 at 3B), CARL pulls ahead of Search-R1 PPO (Figure 2(b)) and the gap widens monotonically. Before this point, segment-level advantages are noisy enough that token-level credit performs comparably; after it, they reliably point in the right direction. Explained variance rises from 0.52 after warm-up to 0.79 at convergence (Appendix I.2, Table 13). Warm-up amplifies the gap monotonically: CARL leads Search-R1 PPO at every warm-up level (cold, random, curated), and the margin grows from $+1.5$ to $+5.1$ EM on HotpotQA at 7B as calibration improves (Appendix H).

Calibration drives emergent selectivity. Because $V(s_0)$ is high on Tier 2 questions, the invoke advantage on those questions is small by construction, and the policy receives a stronger gradient from answering directly. This is the mechanism producing the gap in Figure 2(c) and the convergence

numbers in Table 3: at 7B, CARL invokes tools on 38.6% of Tier 2 questions versus 87.5% for Search-R1 GRPO, despite using no explicit uncertainty heuristic, indicating that selectivity emerges from $V(s_0)$, a learned competence estimate. Despite calling tools far less often, CARL achieves 87.0 Tier 2 EM compared to 77.2 for Search-R1 PPO and 75.1 for GRPO, achieving fewer tool calls and higher accuracy on the same questions. CARL also reduces hallucination on Tier 1 questions, where $V(s_0)$ consistently routes to tool use rather than fabrication (Appendix N).

The 3B selectivity peak is higher and later because the less sharp $V(s_0)$ (AUC 0.85) provides less counter-pressure; 3B gains exceed 7B at every hop count (Appendix M, J).

4.5 What Bounds CARL's Gains

Manual analysis on Musique (Appendix F) finds that 48% of CARL's remaining errors trace to irrelevant BM25 retrieval, with another 14% to query formulation; combined, retrieval-side failures account for 62% of remaining errors, with the dominant category (irrelevant retrieval) stable across hop counts (47–50%). Replacing BM25 with a commercial search API (Appendix G) lifts CARL's Musique accuracy from 30.4 to 45.0 EM, bringing a 7B open-weight model into the range typically reported for substantially larger models on Musique. These results suggest retrieval quality, not the credit-assignment method, sets the ceiling on Musique.

5 Limitations

Warm-up cost. Segment-level credit requires a reasonably calibrated critic before PPO. Our warm-up procedure (§3.3) mitigates the cold-start problem, but an initial training phase, during which the critic learns from base-model rollouts without policy updates, remains an inherent overhead. Moreover, the warm-up ablation (Appendix H) shows the benefit of curated warm-up scales with model size: at 3B even random-data warm-up suffices for CARL to lead Search-R1 PPO, while at 7B curated warm-up is needed to realize the full $+5.1$ EM margin (Appendix H Table 9); CARL still leads at every warm-up level, but the margin shrinks from $+5.1$ to $+1.5$ EM as warm-up degrades.

Segment granularity. All tokens within a segment share one advantage value, so intra-segment variation (e.g., a partly useful extraction) is not captured; credit is uniform within each segment even when individual tokens contribute unequally.

Computational overhead. Each tool call adds two segments and corresponding critic forward passes. Current serving stacks (vLLM, TGI, SGLang) are optimized for uninterrupted decode-to-EOS and do not natively support mid-generation pause, K/V cache surgery, or synchronous tool calls, so segment boundaries currently require per-segment round-trips and partial re-prefill. This is an engineering gap in the serving layer rather than a methodological one, and we expect it to narrow as inference engines add first-class tool-call primitives.

6 Conclusion

Tool-use trajectories have structurally observable boundaries that pure reasoning trajectories lack. CARL exploits this structure: a critic trained on the model's own rollouts turns binary outcome reward into per-segment advantages grounded in the SMDP Bellman equation, giving each tool interaction its own credit without external judges or step-level annotations. The result is a model that learns not just *how* to use tools but *when*, issuing fewer unnecessary calls, formulating better queries, and extracting more faithfully, with gains concentrating on multi-hop tasks where credit assignment is hardest.

Several directions follow naturally. The invoke/assimilate/commit decomposition applies unchanged to richer tool ecosystems (code execution, API calls, multi-agent coordination) where trajectory-level credit becomes even less informative. The competence boundary encoded in $V(s_0)$ could serve as a routing signal at inference time, dispatching to specialized tools or larger models only when predicted success probability is low. More broadly, our finding that smaller models benefit disproportionately suggests that segment-level credit assignment is a practical alternative to scaling parametric memory for tail-distribution knowledge, particularly relevant for resource-constrained settings such as on-device assistants and local coding agents where smaller models with strong tool-use behavior offer a viable alternative to API-bound large models.

References

[1] Arjona-Medina, J.A., et al. (2019). RUDDER: Return Decomposition for Delayed Rewards. NeurIPS.

[2] Kumar, A., Kumar, N., & Gupta, S. (2026). Execution-Grounded Credit Assignment for GRPO in Code Generation. arXiv:2603.16158.

[3] Taparia, A., et al. (2026). ARC: Learning to Configure Agentic AI Systems. arXiv:2602.11574.

[4] Zhang, Y. (2025). Agent-as-Tool: A Study on the Hierarchical Decision Making with Reinforcement Learning. arXiv:2507.01489.

[5] Wang, H., Xu, Q., Liu, C., Wu, J., Lin, F., & Chen, W. (2025). HICRA: Emergent Hierarchical Reasoning in LLMs through Reinforcement Learning. arXiv:2509.03646.

[6] Jin, B., et al. (2025). Search-R1: Training LLMs to Reason and Leverage Search Engines with RL. COLM 2025.

[7] Li, X., Zou, H., & Liu, P. (2025). ToRL: Scaling Tool-Integrated RL. arXiv:2503.23383.

[8] Ng, A.Y., Harada, D., & Russell, S. (1999). Policy Invariance Under Reward Transformations. ICML.

[9] Qian, C., et al. (2025). ToolRL: Reward is All Tool Learning Needs. NeurIPS 2025.

[10] Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2016). High-Dimensional Continuous Control Using Generalized Advantage Estimation. ICLR.

[11] Schulman, J., et al. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.

[12] Sutton, R.S., Precup, D., & Singh, S. (1999). Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in RL. Artificial Intelligence 112, 181-211.

[13] Setlur, A., et al. (2025). Rewarding Progress: Scaling Automated Process Verifiers for LLM Reasoning. ICLR 2025.

[14] Guo, Y., Xu, L., Liu, J., Ye, D., & Qiu, S. (2025). Segment Policy Optimization: Effective Segment-Level Credit Assignment in RL for Large Language Models. NeurIPS 2025.

[15] Kazemnejad, A., et al. (2024). VinePPO: Unlocking RL Potential For LLM Reasoning Through Refined Credit Assignment. arXiv:2410.01679.

[16] Cobbe, K., et al. (2021). Training Verifiers to Solve Math Word Problems. arXiv:2110.14168.

[17] Yang, Z., et al. (2018). HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. EMNLP.

[18] Ho, X.N., Duong Nguyen, A.K., Sugawara, S., & Aizawa, A. (2020). Constructing A Multi-Hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. COLING.

[19] Chen, Z., et al. (2021). FinQA: A Dataset of Numerical Reasoning over Financial Data. EMNLP.

[20] Trivedi, H., Balasubramanian, N., Khot, T., & Sabharwal, A. (2022). MuSiQue: Multihop Questions via Single Hop Question Composition. TACL.

[21] Chen, J., et al. (2025). ReSearch: Learning to Reason with Search for LLMs via Reinforcement Learning. arXiv:2503.19470.

[22] Li, Y., et al. (2025). R3-RAG: Learning Step-by-Step Reasoning and Retrieval for LLMs via Reinforcement Learning. arXiv:2505.23794.

[23] Zheng, R., Dou, S., Gao, S., Hua, Y., Shen, W., et al. (2023). Secrets of RLHF in Large Language Models Part I: PPO. arXiv:2307.04964.

[24] Hu, J., et al. (2024). OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework. arXiv:2405.11143.

[25] Yu, Q., et al. (2025). DAPO: An Open-Source LLM Reinforcement Learning System. arXiv:2503.14476.

[26] Yao, S., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.

[27] Yu, Y., et al. (2024). StepTool: A Step-Level Reinforcement Learning Method for Tool Learning. arXiv:2410.07745.

A SMDP Derivation Details

From the SMDP Bellman equation. By Theorem 1 of Sutton et al. (12), our setting is an SMDP over segment-level decision points, with per-segment TD error

$$\delta_k^{\text{SMDP}} = r_k + \gamma^{\tau_k} V(s_{k+1}) - V(s_k), \tag{4}$$

where r_k is the cumulative reward during segment k and τ_k is its length in tokens. Two properties of our setting simplify this expression. First, the task is episodic with finite horizon (at most 15 segments), so we set $\gamma = 1$ and $\gamma^{\tau_k} = 1$ for every segment. Second, reward is terminal-only: $r_k = 0$ for $k < N$ and $r_N = R$. Substituting into Equation 4 gives the per-segment advantage in Equation 1.

Total credit is preserved. Summing the per-segment advantages along a trajectory, the intermediate V_ϕ terms cancel pairwise, leaving only the endpoints:

$$\sum_{k=0}^{N} A(\text{seg}_k) \;=\; \big[V_\phi(s_1) - V_\phi(s_0)\big] + \big[V_\phi(s_2) - V_\phi(s_1)\big] + \cdots + \big[R - V_\phi(s_N)\big] \;=\; R - V_\phi(s_0). \tag{5}$$

This is the same total credit that trajectory-level PPO would assign from the starting state. Segment-level credit does not add or remove credit; it redistributes the same total across segments, crediting or penalizing each according to how much it changed the critic's estimate. The comparison between CARL and trajectory-level baselines is therefore a comparison of *how* credit is distributed, not *how much*.[4]

Why $\lambda = 0$. One could apply GAE at the segment level: $A_k^{\text{GAE}} = \delta_k + \lambda\delta_{k+1} + \lambda^2\delta_{k+2} + \cdots$. We use $\lambda = 0$ (the one-step TD error) for two related reasons that together make the choice structural rather than empirical.

First, segment-level credit assignment fundamentally relies on isolating the contribution of each segment; multi-step lookahead reintroduces the temporal smearing that segment-level credit is designed to avoid. A bad invoke followed by a strong recovery would see its negative advantage diluted by future positive residuals, defeating the purpose of per-segment attribution.

The $\lambda \to 1$ limit collapses to trajectory-level credit. Applying GAE (10) at the segment level with discount $\gamma = 1$ gives

$$A^{\text{GAE}}(\text{seg}_k) \;=\; \sum_{l=0}^{N-k} \lambda^l \, \delta_{k+l}, \qquad \delta_j = r_j + V_\phi(s_{j+1}) - V_\phi(s_j). \tag{6}$$

Under terminal-only reward ($r_j = 0$ for $j < N$, $r_N = R$, with $V_\phi(s_{N+1}) \equiv 0$), this gives $\delta_j = V_\phi(s_{j+1}) - V_\phi(s_j)$ for $j < N$ and $\delta_N = R - V_\phi(s_N)$. At $\lambda = 1$, Eq. 6 becomes a telescoping sum:

$$\begin{aligned} A^{\text{GAE}}(\text{seg}_k)\big|_{\lambda=1} \;&=\; \sum_{l=0}^{N-k} \delta_{k+l} \\ &=\; \underbrace{\big[V_\phi(s_{k+1}) - V_\phi(s_k)\big]}_{\delta_k} + \underbrace{\big[V_\phi(s_{k+2}) - V_\phi(s_{k+1})\big]}_{\delta_{k+1}} + \cdots + \underbrace{\big[R - V_\phi(s_N)\big]}_{\delta_N} \\ &=\; R - V_\phi(s_k). \end{aligned} \tag{7}$$

Every intermediate V_ϕ term cancels pairwise; only the endpoints $-V_\phi(s_k)$ and R survive.

Equation 7 is the structural reason CARL requires $\lambda = 0$. At $\lambda = 1$, every segment's advantage reduces to the same functional form $R - V_\phi(s_k)$ — the trajectory's outcome reward minus the critic's pre-segment estimate. Because $V_\phi(s) \in [0, 1]$ (sigmoid output, Appendix D Table 5):

[4] The intermediate advantage $V_\phi(s_{k+1}) - V_\phi(s_k)$ has the potential-based form $\Phi(s') - \Phi(s)$ of Ng et al. (8), with the critic V_ϕ playing the role of the potential Φ. Because the potential is learned jointly with the policy rather than fixed in advance, the full policy-invariance guarantee holds only as V_ϕ stabilizes; but the form motivates our construction and matches the shape of process rewards used in prior work (§2).

- In a successful trajectory ($R = 1$), $A(\text{seg}_k) = 1 - V_\phi(s_k) \geq 0$ for every k. Every segment receives non-negative credit.
- In a failed trajectory ($R = 0$), $A(\text{seg}_k) = -V_\phi(s_k) \leq 0$ for every k. Every segment receives non-positive credit.

This is exactly trajectory-level credit applied at segment-level decision points: structurally equivalent to a coarser-granularity Search-R1 PPO. The within-trajectory sign divergence reported in Appendix I.1 (positive invoke advantage paired with negative assimilate advantage in the *same* trajectory) becomes mathematically impossible, because both segments' advantages share the sign of R.

The mechanism that distinguishes CARL from trajectory-level credit therefore requires $\lambda < 1$. Appendix H.1 confirms this empirically: as λ rises from 0 to 1, CARL's HotpotQA EM falls from 47.9 to 42.5, converging to Search-R1 PPO's 42.8.

The variance cost of $\lambda = 0$ is small in our setting: bounded binary rewards, short horizons (≤ 15 segments), and MC-trained critic targets (Appendix C) keep per-segment V_ϕ predictions calibrated, so the one-step TD error has acceptable variance. The credit-precision benefit is large.

B Partially-Observed SMDP Formalism

In CARL's default context-management mode (code replacement + assimilation), the raw tool output o_k^{raw} is observed by the assimilate segment but removed from context before the next segment begins. This section formalizes the resulting partially-observed SMDP and shows that segment-level advantages remain well-defined.

Transient and persistent states. Let s_k denote the *persistent* state at segment boundary k, i.e. the full context visible to all subsequent segments. After an invoke segment, the rollout machinery appends the code output, producing a *transient* state

$$\tilde{s}_{k+1} = s_k \oplus t_k \oplus o_k^{\text{raw}}, \tag{8}$$

where t_k is the model-generated text (planning + code) and o_k^{raw} is the raw stdout. The assimilate segment conditions on $\tilde{s}_{k+1}$ and produces an extraction e_{k+1}. The rollout loop then replaces o_k^{raw} with e_{k+1}, yielding the next persistent state:

$$s_{k+2} = s_k \oplus t_k \oplus e_{k+1}. \tag{9}$$

Observation model. The mapping $\tilde{s}_{k+1} \mapsto s_{k+2}$ is *not* a fixed function of the environment; it depends on the policy's assimilation output e_{k+1}. This means the transition from boundary k to boundary $k+2$ is partially observed: the invoke segment's raw output is visible only to the immediately following assimilate segment, not to any later segment.

Segment-level advantages remain well-defined. The critic V_ϕ is evaluated at persistent-state boundaries $s_0, \tilde{s}_1, s_2, \tilde{s}_3, s_4, \ldots$ (alternating transient and persistent for tool-call trajectories). Each advantage $A(\text{seg}_k) = V_\phi(s_{k+1}) - V_\phi(s_k)$ depends only on the two boundary states of that segment, both of which are observed by the critic at evaluation time. The partial observability affects what information future segments can condition on, but not the well-definedness of the advantage at the segment where V_ϕ is computed. The telescoping property (Equation 5) holds identically: intermediate V_ϕ terms cancel regardless of whether boundary states are transient or persistent.

Why gradient masking is unnecessary. In Search-R1 and ToRL, environment-injected tokens (tool output) appear in the generation context and must be masked from the policy gradient to avoid reinforcing tokens the model did not produce. In CARL, raw tool output o_k^{raw} is replaced by e_{k+1} before the next generation begins, so the only tokens in the policy's generation target are model-produced tokens. Each segment's loss (Equation 3) sums over g_k, the model-generated tokens in that segment; environment tokens never enter g_k. Gradient masking is therefore unnecessary by construction.

C Warm-up Implementation Details

Dataset sizes per bucket. The warm-up dataset is constructed from base-model rollouts on the training split. Table 4 reports the number of trajectories per bucket.

Table 4: Warm-up dataset composition (number of trajectories). Both scales use 32K trajectories total, but the Tier 1/Tier 2 split differs because it is determined by each base model's competence: the 7B model answers more questions correctly without tools (45% Tier 2), while the 3B model fails more often (35% Tier 2). Within each tier, trajectories are split equally between no-tool and forced-tool rollouts.

Bucket	7B	3B
Tier 2, no-tool (anchors $V(s_0) \approx 1$)	7,200	5,600
Tier 2, forced-tool (unnecessary tool risk)	7,200	5,600
Tier 1, no-tool (anchors $V(s_0) \approx 0$)	8,800	10,400
Tier 1, forced-tool (teaches assimilation)	8,800	10,400
Total	32,000	32,000

The Tier 2/Tier 1 ratio reflects each base model's competence boundary: 45/55 at 7B and 35/65 at 3B (averaging $\sim$40/60 across scales), with roughly 70% BM25 retrieval and 30% high-quality search API retrieval within each bucket.

Curation principles. The warm-up set is built from base-model rollouts, labeled by final correctness. No human annotation and no external judge are used.

1. **Tier 1 / Tier 2 labeling via multi-rollout consistency.** A question is Tier 2 (within parametric competence) if the base model answers it correctly in at least one of five no-tool rollouts, and Tier 1 (beyond parametric competence) if all five rollouts produce incorrect answers. This labeling directly measures the model's competence boundary: Tier 2 questions are those the model can sometimes answer from its own knowledge; Tier 1 questions are those where it consistently cannot. Because the 7B model answers more questions correctly, its warm-up set is 45% Tier 2; the 3B set is 35% Tier 2. Both force V_ϕ to discriminate along the competence axis.
2. **Controlled tool-use rollouts.** Each question is also rolled out under two system prompts: (a) forced tool use, (b) no tools allowed. Combined with the Tier labels, this produces four outcome buckets per question, each teaching a different signal: Tier 2 no-tool (anchors $V(s_0) \approx 1$), Tier 2 forced-tool (teaches that unnecessary tools add risk), Tier 1 no-tool (anchors $V(s_0) \approx 0$), Tier 1 forced-tool (teaches assimilation).
3. **Mixed retrieval quality.** Roughly 70% BM25 (matching PPO training) and 30% rollouts from a high-quality internal search API give V_ϕ contrast between helpful and less-helpful tool outputs without changing the inference-time retriever.
4. **Topic diversity via embedding clustering.** Questions are embedded with a sentence-transformer (all-MiniLM-L6-v2) and clustered with k-means; warm-up samples are drawn uniformly across clusters to prevent V_ϕ from learning surface topical cues.
5. **Multi-hop exposure.** Multi-hop rollouts from 2WikiMQA and Musique are included so that V_ϕ has seen compound-state boundaries before PPO.

Verification gate. Before starting PPO, three checks must pass on a held-out subset.

Minimum thresholds (gate): (i) $V(s_0)$ separates Tier 1 from Tier 2 questions (AUC ≥ 0.70), (ii) V_ϕ shows correct sign behavior after retrieval on forced-tool rollouts ($\geq 60\%$ of cases), and (iii) the critic achieves sufficient explained variance (EV ≥ 0.45).

Target thresholds (well-warmed-up critic): A well-warmed-up critic typically clears the gate by a comfortable margin (AUC ≥ 0.80, sign accuracy $\geq 70\%$, EV ≥ 0.55). Random-data warm-up alone reaches AUC 0.72, only just above the 0.70 gate, so the gate is necessary but not sufficient: calibration quality (ECE) matters beyond ranking ability.

What the ablations show. The three warm-up variants produce qualitatively different critic shapes. Full warm-up yields a near-diagonal calibration curve (ECE = 0.043), random-data warm-up produces a compressed monotone curve (ECE = 0.137), and cold-start gives a flat near-chance line (ECE = 0.238). Figure 2(a) plots predicted success probability $V(s_0)$ against actual success rate on held-out questions. Full warm-up tracks the diagonal closely; random-data warm-up ranks correctly but pulls predictions toward the mean (slope ≈ 0.4), concentrating calibration error at the extremes; cold-start predictions are nearly flat, consistent with AUC ≈ 0.54 (near chance). Figure 2(b) plots held-out evaluation accuracy throughout training: curated warm-up pulls ahead of Search-R1 PPO within the first hundred steps, random-data warm-up lags, and cold-start plateaus below both.

System prompt templates. Four system prompts are used during warm-up data collection:

1. **No-tool direct answer:** "Answer the following question directly. Do not use any tools or code. Provide your answer inside \boxed{}."
2. **Forced tool use:** "You must use a Python code block to help answer this question. Use `search(query)` for retrieval or write arithmetic code. After seeing tool output, write a `<context>` block extracting the relevant information, then provide your answer inside \boxed{}."
3. **Optional tool use (Tier 2):** Same as forced tool use but with "You may optionally use" replacing "You must use."
4. **Multi-hop forced tool use:** Same as (2) but with an additional instruction: "This question may require multiple search steps. You may call tools more than once."

Embedding clustering. Questions are embedded with `all-MiniLM-L6-v2` (384-dimensional embeddings). We apply k-means with $k = 50$ clusters for HotpotQA and 2WikiMQA, $k = 30$ for GSM8K (smaller and more homogeneous). Warm-up samples are drawn uniformly across clusters within each bucket to prevent the critic from learning surface topical cues. Clustering is performed once on the full training split; the same cluster assignments are reused for both scales.

Training schedule. Warm-up training continues until the verification gate passes on a held-out subset: AUC ≥ 0.70, sign accuracy ≥ 0.60, and explained variance ≥ 0.45. We checkpoint every 25 steps and use the first checkpoint at or after step 1,800 that clears the gate. In practice this requires 1,800–2,400 critic-only steps depending on scale, and all 6 trained models (2 scales $\times$ 3 seeds) cleared the gate within this range. Each step processes a batch of 256 (trajectory, boundary) state-target pairs sampled uniformly across the four warm-up buckets, with MSE loss against Monte-Carlo terminal returns. This corresponds to roughly 5–6 epochs over the boundary-state pool ($\sim$96K boundaries from 32K trajectories at $\sim$3 boundaries each). The optimizer is AdamW (β_1=0.9, β_2=0.999) with weight decay 0.0 for the value head and 0.01 for the backbone. We set value-head weight decay to zero because L2 regularization on a sigmoid output shrinks predictions toward 0.5, undermining calibration. Learning rates are 5e-6 for the value head and 5e-7 for the backbone ($10\times$ smaller, to preserve policy representations), with linear warmup over the first 100 steps and constant thereafter. Gradient clipping is set at $\|\nabla\|_2 = 1.0$. Policy weights are frozen; only the value head and shared backbone receive gradients.

Example warm-up rollouts. Representative trajectories from each bucket, showing the critic's V_ϕ at each segment boundary, are provided in the supplementary materials.

D Training Hyperparameters and Compute

Table 5 reports the full training configuration. CARL is implemented in OpenRLHF (24); Search-R1 PPO is trained with the verl-based public codebase released by Jin et al. (6). Shared algorithmic values are matched across both frameworks; framework-specific names are given in parentheses for reproducibility.

Step accounting and data exposure. We use the term *PPO step* (or simply *step*) to mean one outer iteration: ROLLOUT (sample 256 prompts and generate n=5 trajectories each, for 1,280 trajectories per step) $\rightarrow$ REWARD/ADVANTAGE (EM check, critic evaluation at boundaries, per-segment advantages) $\rightarrow$ LEARN (4 PPO inner epochs over the 1,280 trajectories; with mini-batch

64, this is 80 gradient updates per step). Across 500 steps, training therefore processes 128K unique prompts (0.75 epoch over the 170K training pool), generates 640K trajectories total, and performs 40K gradient updates. The 0.75-epoch coverage is in line with published RL-LLM practice (6; 25) where prompt-level sample efficiency is governed by gradient noise rather than data exhaustion.

Table 5: Training hyperparameters. CARL implemented in OpenRLHF; Search-R1 PPO baseline reproduced from its public verl codebase. Where a name differs across frameworks, both are shown.

Hyperparameter	CARL (OpenRLHF)	Search-R1 PPO (verl)
PPO algorithm		
PPO clip ratio (`eps_clip`)	0.2	0.2
GAE λ (`lambd`)	0.0 (one-step TD, §3.2)	0.95
Discount γ	1.0	1.0
Value loss coef (`vf_coef`)	0.5	0.5
Entropy coef	0.001	0.001
KL penalty coef (`init_kl_coef`)	0.001	0.001
KL estimator	k3 (`use_kl_estimator_k3`)	low_var_kl
KL as loss term	`use_kl_loss=true`	`use_kl_loss=true`
Batch sizes		
Rollout batch (`rollout_batch_size`, prompts/step)	256	256 (`data.train_batch_size`)
Micro-rollout batch per GPU (`micro_rollout_batch_size`)	8	— (dynamic)
Train batch (`train_batch_size`, PPO mini-batch)	64	64 (`ppo_mini_batch_size`)
Micro-train batch per GPU (`micro_train_batch_size`)	2	2 (`ppo_micro_batch_size_per_gpu`)
Rollouts per prompt (`n_samples_per_prompt`)	5	5 (`rollout.n`)
Trajectories per RL step	1,280	1,280
PPO inner epochs (`max_epochs`)	4	4 (`ppo_epochs`)
Sequence lengths		
Prompt max len (`prompt_max_len`)	2048	2048 (`max_prompt_length`)
Generate max len (`generate_max_len`)	2048	2048 (`max_response_length`)
Max segments per episode (CARL only)	15	—
$T_{\text{assimilate}}$	256	—
Generation (rollout sampling)		
Temperature	1.0	1.0
Top-p	1.0	1.0
Top-k	-1 (off)	-1
Rollout backend	vLLM (`-vllm_num_engines 4`)	vLLM (`rollout.name=vllm`)
Learning rates (constant after 100-step linear warm-up)		
Actor LR (`actor_learning_rate`)	1e-6	1e-6
Critic value-head LR	5e-6	1e-5 (`critic.optim.lr`)
Critic backbone LR	5e-7	— (frozen)
Optimization		
Optimizer	AdamW	AdamW
(β_1, β_2)	$(0.9, 0.999)$	$(0.9, 0.999)$
Weight decay (head / backbone)	0.0 / 0.01	0.0 / 0.01
Gradient clip (`max_norm`)	1.0	1.0
Mixed precision	bf16 (`-bf16`)	bf16
Memory & sharding		
ZeRO stage	3 (`-zero_stage 3`)	3 (FSDP equivalent)
Adam offload	enabled (`-adam_offload`)	enabled
Gradient checkpointing	enabled	enabled
Flash attention	enabled (`-flash_attn`)	enabled
Critic architecture (CARL only)		
Value head	2-layer MLP	1-layer linear (default)
Hidden dim	3584 (7B) / 2048 (3B)	d_{model}
Activation	GELU	—
Output activation	sigmoid $\rightarrow [0, 1]$	identity
Final layer init	zero	zero
Head trainable params	12.85M (7B) / 4.20M (3B)	3.6K (7B) / 2.0K (3B)
Schedule		
Critic warm-up steps (CARL only)	1,800–2,400 (gate-passing)	—
PPO outer steps (`num_episodes`)	500	500
Total prompts processed	128K ($\sim$0.75 epoch)	128K ($\sim$0.75 epoch)
Total trajectories generated	$\sim$640K	$\sim$640K
Total gradient updates	40K	40K
Eval frequency (`-eval_steps`)	every 25 steps	every 25 steps (`test_freq`)
Compute (per training run, 8$\times$ NVIDIA H100 80GB)		
Wall-clock 7B	$\sim$50 hours	$\sim$49 hours
Wall-clock 3B	$\sim$22 hours	$\sim$21 hours
GPU-hours 7B	$\sim$400 H100-hours	$\sim$390 H100-hours
GPU-hours 3B	$\sim$175 H100-hours	$\sim$170 H100-hours
Reproducibility		
Random seeds	$\{1, 2, 3\}$	$\{1, 2, 3\}$
Training runs	6 (3 seeds $\times$ 2 scales)	6 (3 seeds $\times$ 2 scales)

Key design choices.

- $\lambda = 0$ **(one-step TD).** Multi-step lookahead would reintroduce the temporal smearing that segment-level credit is designed to avoid. With short horizons (≤ 15 segments), bounded binary rewards, and MC-trained critic targets (Appendix C), the variance cost of $\lambda = 0$ is small while the credit-precision benefit is large.
- **Critic backbone trainable, LR $10\times$ smaller.** Search-R1's critic is a single linear head on top of the frozen reference model, so its critic capacity is bounded by the pretrained representations. CARL trains the backbone with LR 5×10^{-7} ($10\times$ smaller than the value head) so that V_ϕ can develop competence-relevant features without destabilizing the policy that shares it.
- **Sigmoid output for V_ϕ.** The terminal reward is binary $R \in \{0, 1\}$ and we train with MSE; a sigmoid-bounded output ensures V_ϕ is a proper success probability and prevents extrapolation outside $[0, 1]$ during warm-up.
- **Value-head weight decay 0.** Weight decay on the head shrinks V_ϕ predictions toward 0.5 and undermines calibration; we disable it for the head and keep standard 0.01 on the backbone.
- **Rollouts-per-prompt $n = 5$.** Matches Search-R1's published verl setting, so all RL methods in our comparison see the same trajectory budget per prompt.

Total compute. All training was conducted on Azure Machine Learning (AML) with access to 4 nodes of $8\times$ NVIDIA H100 80GB each (32 GPUs total), enabling up to 4 training runs in parallel. Evaluation, inference, and Tier-1/Tier-2 label construction were performed on AMD MI300X GPUs; all GPU-hours below are translated to single-H100-equivalent hours for easy comparison. The reported main-table runs (3 seeds $\times$ 4 method-scale combinations $=$ 12 training runs) consume approximately 3,400 H100-hours. Single-run baselines (Search-R1 GRPO at both scales, CoT+Tools evaluation) add $\sim$575 H100-hours. Reported ablations (cold-start and random-data warm-up, Appendix H; commercial-search experiment, Appendix G) add $\sim$1,500 H100-hours. The total compute consumed for *all reported numbers in this paper is approximately 5,500 H100-hours*. With 4 nodes available, the 12 main-table runs completed in roughly 2 weeks of wall-clock time. Substantial additional compute went into supporting work: building the Tier-1/Tier-2 labels via 5-rollout consistency on the full training pool ($\sim$5,000 H100-hours), prompt template iteration during early development, debugging failed runs, and analysis rollouts for Figures 3 and 4; together these account for roughly $3\times$ the reported budget. **The total compute budget for the project is approximately 22,000 H100-hours.**

E Tool Call Frequency and Deployment Cost

While Table 2 reports per-episode *token counts* for every method, it does not report *tool call counts*. Tool calls are independently meaningful for two reasons: (i) at deployment, search and code-execution APIs incur per-call latency and dollar cost beyond what token-count alone captures, and (ii) tool-call counts disambiguate whether CARL's lower token usage comes from selective invocation (good) or from truncated reasoning (bad).

Table 6: Average tool calls per episode (number of `invoke` segments). Tier 1 questions are nominally always-invoke for search benchmarks, so the upper bound for HotpotQA/2WikiMQA is the dataset's hop count ($\sim$2). CARL invokes fewer tools than Search-R1 PPO at every dataset and scale, while remaining within the dataset's hop budget on Tier 1 questions (Table 3).

Scale	Method	GSM8K	HotpotQA	2WikiMQA	FinQA	Musique
7B	Search-R1 GRPO	—	1.94	1.86	—	3.51
	Search-R1 PPO	—	1.91	1.84	—	3.48
	CARL (ours)	**0.04**	**1.57**	**1.21**	**2.32**	**3.28**
3B	Search-R1 GRPO	—	1.96	1.91	—	3.58
	Search-R1 PPO	—	1.93	1.88	—	3.55
	CARL (ours)	**0.07**	**1.65**	**1.32**	**2.41**	**3.41**

Per-dataset patterns. The ideal number of tool calls depends on the dataset: GSM8K questions are solvable without tools (97% Tier 2), so the optimal count is near zero; HotpotQA and 2WikiMQA are 2-hop but contain a mix of Tier 1 and Tier 2, so the optimal count is well below 2; Musique is 3–4

hop and almost entirely Tier 1 (93%), so most questions genuinely need every hop. CARL matches these expectations: 0.04 calls on GSM8K (calculator skipped 96% of the time), 1.57 and 1.21 on HotpotQA and 2WikiMQA (vs. $\sim$1.9 for both baselines, which search indiscriminately), and 3.28 on Musique (vs. 3.48, a small gap because there is little to skip). In short, CARL learns to call tools only when they are needed.

Deployment cost. For HotpotQA-style queries at 7B, CARL generates 1,193 tokens per episode (Table 2) and issues 1.57 tool calls (Table 6), versus Search-R1 PPO's 1,837 tokens and 1.91 calls, a 35.1% token reduction and 17.8% fewer tool invocations. Assuming 200 ms per-call search latency (typical for warm-cache BM25 and many commercial search APIs), the per-query latency reduction is approximately 20.1%. The token saving translates to a proportional reduction in serving cost on token-priced LLM APIs; the tool-call saving translates to a proportional reduction in retrieval-side cost (search-API charges, cache pressure, network round-trips). On Musique the tool-call savings shrink (3.28 vs. 3.48, -5.7%) because the dataset is dominated by Tier 1 questions where CARL must invoke at every hop; on this dataset, deployment savings come almost entirely from token reduction (6.4% fewer tokens).

F Musique Error Analysis

We manually analyzed 100 incorrect CARL predictions on Musique at 7B. Errors were sampled uniformly at random from the 348 incorrect predictions in our 500-question Musique evaluation slice ($1 - 0.304$ EM $\times$ 500). Two annotators independently categorized each error using a written rubric; inter-annotator agreement was 87% (Cohen's $\kappa = 0.81$). Disagreements (13 cases) were resolved by discussion.

Sample composition. Musique (20) consists of 2-hop, 3-hop, and 4-hop questions in roughly a 55/30/15 dev-set ratio. Per-hop EM rates of 40%/22%/10% (declining with hops, as expected) approximately reproduce the headline 30.4% Musique EM under this distribution (computed value 30.1%; the small gap reflects rounding). Correspondingly, our 100 sampled errors break down to 45 from 2-hop, 35 from 3-hop, and 20 from 4-hop questions.

Table 7: Error categories on 100 incorrect Musique predictions (CARL 7B), with per-hop disaggregation.

Error Category	2-hop (n=45)	3-hop (n=35)	4-hop (n=20)	Total
Irrelevant retrieval (BM25 returns unhelpful passage)	21 (47%)	17 (49%)	10 (50%)	48
Query formulation error	2 (4%)	6 (17%)	6 (30%)	14
Assimilation error (`<context>` drops fact)	10 (22%)	5 (14%)	1 (5%)	16
Multi-hop reasoning error	6 (13%)	4 (11%)	2 (10%)	12
Commit error (correct context, wrong answer)	4 (9%)	2 (6%)	1 (5%)	7
Other	2 (4%)	1 (3%)	0 (0%)	3

Headline finding. 48% of remaining errors are *irrelevant retrieval*: the BM25 retriever returns an unhelpful passage even when CARL has formulated a reasonable query for that hop. This share is roughly stable across hop counts (47%–50%), indicating a scale-independent BM25 failure rate. It motivates the commercial-search experiment in Appendix G, where replacing BM25 with a higher-quality retrieval API lifts CARL's Musique accuracy further.

Other categories. Assimilation errors (16%) are the category CARL's segment-level credit directly addresses; they account for a meaningful but bounded share of remaining failures. Multi-hop reasoning and commit errors (12% + 7% = 19%) are categories where the model has the right facts but composes them incorrectly; no credit-assignment scheme directly addresses these. Query-formulation errors (14%) are concentrated at higher hop counts, where each query depends on facts extracted from prior hops. At 3B we observe a similar irrelevant-retrieval share ($\sim$47%) but a somewhat higher query-formulation share ($\sim$21% vs. 14% at 7B), consistent with smaller models having less robust query composition; other categories track 7B within 2–3 percentage points.

Sample failed trajectories. **Example 1: Irrelevant retrieval.** Question: "What is the capital of the country where [entity] was born?" The model correctly decomposes this into two hops but the BM25 retriever returns a passage about a different person with a similar name. The critic correctly assigns negative invoke advantage (-0.31) to the second hop.

Example 2: Assimilation error. Question: "When was the director of [film] born?" The model retrieves a passage containing both the director's name and birth year, but the `<context>` block extracts only the director's name, losing the birth year. Invoke advantage: $+0.28$; assimilate advantage: -0.22.

Example 3: Multi-hop reasoning error. Question: "What language is spoken in the country where [river] originates?" The model correctly retrieves the river's origin country and the country's languages, but selects the colonial-era language rather than the current official language. Both invoke advantages are positive; the commit advantage is negative (-0.41).

G Retrieval Ablation

We vary the retrieval setup at inference time while holding the trained CARL 7B model fixed. All configurations are evaluated on the full 500-question Musique dev slice.

Table 8: CARL 7B Musique EM under varied retrieval setups.

Retrieval setup	EM	Δ vs. default
BM25, $k = 3$ (default; main results)	30.4	—
BM25, $k = 5$	32.5	$+2.1$
BM25, $k = 7$	30.8	$+0.4$
Commercial search API, $k = 3$	**45.0**	$+14.6$

$k = 5$ improves recall modestly. $k = 7$ saturates the assimilation budget ($T_{\text{assimilate}} = 256$), erasing most of the gain. Replacing BM25 with a commercial search API delivers a large lift, bringing CARL 7B into the EM range typically reported for substantially larger models on Musique. This confirms that CARL's query formulation, selective invocation, and assimilation were in fact effective.

H Ablation: Warm-up Variants

Both CARL and Search-R1 PPO use the same curated critic warm-up procedure in our main results (Appendix C). A natural concern is whether CARL's gain over Search-R1 PPO is driven by segment-level credit assignment or by the warm-up itself. We address this directly: we ablate the warm-up for both methods and report the results.

Table 9: Warm-up ablation across both methods (Exact Match). Best CARL row in bold. "Cold start" uses no separate warm-up phase (the standard PPO-LLM default, equivalent to the original Search-R1 setup (6)). "Random-data" uses the same number of warm-up steps but draws rollouts uniformly without the Tier 1/2 curation. "Curated" is the full Appendix C procedure used in the main results.

Method	Warm-up	GSM8K	HotpotQA	2Wiki	FinQA	Musique
7B						
CARL	Cold start	88.7	39.0	39.5	53.0	21.5
CARL	Random-data	90.5	43.0	43.0	56.0	25.5
CARL	**Curated (full)**	**91.5**	**47.9**	**48.3**	**59.3**	**30.4**
SR1-PPO	Cold start	—	37.5	38.0	—	19.0
SR1-PPO	Random-data	—	40.5	40.0	—	20.5
SR1-PPO	Curated (matched)	—	42.8	41.7	—	22.1
3B						
CARL	Cold start	84.3	28.8	32.9	38.7	13.4
CARL	Random-data	86.0	34.7	38.3	42.9	16.8
CARL	**Curated (full)**	**87.2**	**40.4**	**44.7**	**46.6**	**21.1**
SR1-PPO	Cold start	—	28.5	30.4	—	9.5
SR1-PPO	Random-data	—	29.6	31.5	—	10.4
SR1-PPO	Curated (matched)	—	30.2	32.1	—	12.1

Segment-level credit has a baseline edge. Even without warm-up, cold-start CARL outperforms cold-start Search-R1 PPO across all search benchmarks at both scales: $+1.5$ EM on HotpotQA at 7B, $+1.5$ on 2WikiMQA, $+2.5$ on Musique, with similar margins at 3B. Segment-level credit produces a useful signal even before the critic is well-calibrated, because the commit segment's advantage $A(\text{commit}) = R - V_\phi(s_N)$ contains the outcome reward directly, and the invoke and assimilate advantages still register the largest state changes (a passage entering the context, an extraction replacing raw output) even with a noisy V_ϕ.

Warm-up amplifies the edge. The CARL–Search-R1 PPO gap grows monotonically as warm-up quality improves. On 7B HotpotQA: $+1.5$ (cold) $\rightarrow$ $+2.5$ (random) $\rightarrow$ $+5.1$ (curated) — a $3.4\times$ growth in CARL's advantage. The same monotonic pattern holds on 2WikiMQA ($+1.5 \rightarrow +3.0 \rightarrow +6.6$) and Musique ($+2.5 \rightarrow +5.0 \rightarrow +8.3$), and at 3B with larger absolute margins ($+0.3 \rightarrow +5.1 \rightarrow +10.2$ on HotpotQA). Search-R1 PPO's curated warm-up gives it 5.3 EM beyond cold-start ($42.8 - 37.5$), but matched warm-up does not help SR1-PPO catch up to CARL — it lifts the floor of both methods, while CARL's segment-level credit pulls its ceiling further away. This rules out warm-up as the source of CARL's gain: CARL leads at every warm-up level, and the lead grows with calibration quality.

CARL EM tracks critic calibration quality. At 7B on HotpotQA, the three warm-up regimes produce a clean monotonic mapping from $V(s_0)$ AUC (Appendix I) to EM:

Warm-up	$V(s_0)$ AUC	CARL HotpotQA EM
Curated	0.93	47.9
Random-data	0.72	43.7
Cold start	0.54	39.2

Each calibration level maps to a distinct EM level. Search-R1 PPO shows a much smaller gradient ($42.8 \rightarrow 40.5 \rightarrow 37.5$ across the same warm-up regimes), and its EM does not track per-segment V_ϕ accuracy because trajectory-level GAE smooths per-token V_ϕ errors. CARL's EM does track V_ϕ calibration, evidence that segment-level credit converts critic calibration into per-segment advantages that meaningfully steer training, in a way that trajectory-level credit cannot.

Anchoring against the original Search-R1 paper. The cold-start Search-R1 PPO numbers also serve as a sanity check against the published baseline. The original Search-R1 paper (6) uses brief critic warm-up driven by early policy rollouts rather than a separately curated phase (their Section 5.1: "PPO relies on a critic model, which requires several warm-up steps before effective training begins"). Their reported Qwen2.5-7B-Instruct PPO numbers on our shared datasets are 37.0 on HotpotQA and

14.6 on Musique; our cold-start reproductions at 37.5 and 19.0 sit close to these (the small bumps reflect our broader training data: 170K prompts including 2WikiMQA versus their NQ+HotpotQA combination). With curated warm-up, our reproductions reach 42.8 and 22.1, which is favorable to the baseline.

H.1 λ Ablation

CARL uses $\lambda = 0$ (one-step TD) while Search-R1 PPO uses $\lambda = 0.95$ (Appendix D Table 5). To verify this asymmetric choice is structural rather than HP-tuning, we sweep λ in CARL while holding everything else fixed (same warm-up, training pool, and PPO setup).

Table 10: λ ablation for CARL on HotpotQA at 7B, single run per condition. Reference: Search-R1 PPO at the same scale and dataset is 42.8 EM (Table 2). As $\lambda \to 1$, CARL's segment advantages telescope to $R - V_\phi(s_k)$ at every segment, mathematically degenerating to trajectory-level credit at segment decision points; the EM correspondingly collapses to roughly Search-R1 PPO's level.

λ	CARL HQ EM	Δ vs $\lambda = 0$	Δ vs SR1-PPO
0.0 (default)	**47.9**	**—**	+5.1
0.5	45.4	−2.5	+2.6
0.95	43.2	−4.7	+0.4
1.0	42.5	−5.4	−0.3

The table makes the structural argument concrete. At $\lambda = 0.95$ (the value Search-R1 PPO uses), CARL's per-segment attribution is mostly washed out by the GAE smoothing across future segments; performance drops 4.7 EM and lands at roughly Search-R1 PPO's level (+0.4 EM). At $\lambda = 1.0$, every segment in a trajectory receives the same advantage $R - V_\phi(s_k)$, and the within-trajectory sign divergence reported in Appendix I.1 becomes mathematically impossible: the diagnostic case (good invoke + bad assimilate within one rollout) cannot produce opposing signs because both segments share the trajectory's terminal reward. CARL's EM at $\lambda = 1.0$ (42.5) is close to Search-R1 PPO (42.8 ± 0.6, 3-seed), as expected — the two are structurally similar at this limit (note: this ablation is a single run, so the 0.3 EM difference is not statistically meaningful). The 5.4 EM gap between $\lambda = 0$ and $\lambda = 1.0$ is, by construction, the entire benefit of segment-level over trajectory-level credit assignment in our setting.

I Calibration Reliability

Table 11 breaks down the calibration curve in Figure 2(a) into per-decile bins, showing how predicted $V(s_0)$ aligns with actual success rate across both scales and all three warm-up regimes. Expected Calibration Error (ECE) measures the average gap between predicted confidence and actual accuracy; a low ECE means the critic's $V(s_0)$ can be trusted as a probability, which is essential for segment-level advantages to carry the right magnitude, not just the right sign. The summary metrics (ECE 0.043, AUC 0.93) cited in §4.4 are derived from this table.

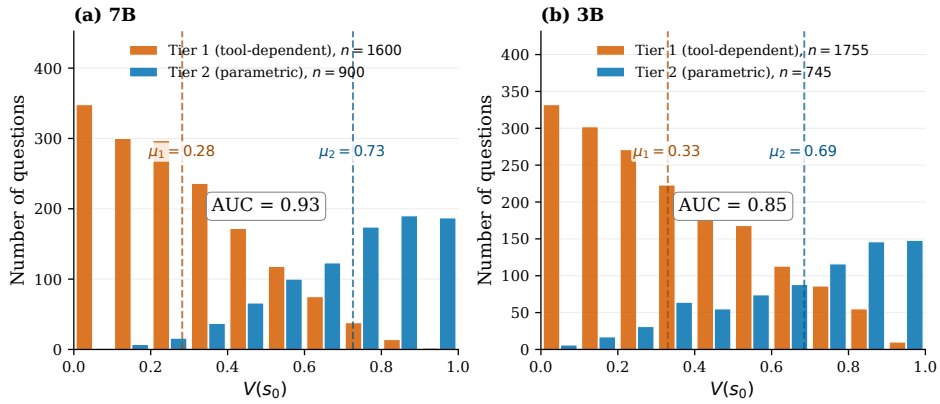


Figure 4: Distribution of $V(s_0)$ for Tier 1 (tool-dependent) and Tier 2 (parametrically solvable) questions after full warm-up. The two distributions are well-separated at both scales (AUC 0.93 at 7B, 0.85 at 3B), confirming that the critic learns to distinguish questions requiring tool use from those the model can answer on its own.

Table 11: Calibration reliability: predicted $V(s_0)$ decile vs. actual success rate (sample count in parentheses). Numbers are aggregated across all 3 CARL seeds; per-seed standard deviations of ECE are within 0.005 across all bins.

	7B			3B		
$V(s_0)$ bin	Curated	Random	Cold	Curated	Random	Cold
0.0–0.1	0.120 (305)	0.304 (175)	0.519 (255)	0.158 (320)	0.331 (188)	0.487 (252)
0.1–0.2	0.182 (285)	0.345 (215)	0.472 (250)	0.221 (290)	0.378 (220)	0.501 (248)
0.2–0.3	0.281 (245)	0.407 (260)	0.487 (252)	0.298 (245)	0.412 (265)	0.476 (253)
0.3–0.4	0.319 (225)	0.384 (290)	0.446 (248)	0.355 (220)	0.405 (285)	0.455 (247)
0.4–0.5	0.381 (195)	0.431 (310)	0.480 (250)	0.428 (195)	0.448 (305)	0.479 (251)
0.5–0.6	0.590 (215)	0.504 (305)	0.507 (245)	0.541 (215)	0.498 (300)	0.514 (246)
0.6–0.7	0.680 (235)	0.530 (280)	0.473 (252)	0.624 (235)	0.535 (280)	0.482 (250)
0.7–0.8	0.686 (270)	0.610 (240)	0.527 (256)	0.692 (270)	0.589 (245)	0.517 (254)
0.8–0.9	0.817 (295)	0.610 (215)	0.502 (248)	0.758 (285)	0.611 (220)	0.493 (247)
0.9–1.0	0.918 (220)	0.635 (210)	0.518 (244)	0.832 (215)	0.628 (215)	0.512 (245)
ECE	0.043	0.137	0.238	0.059	0.147	0.238
Brier score	0.179	0.266	0.328	0.202	0.273	0.328
AUC (T1 vs T2)	0.93	0.72	0.54	0.85	0.65	0.51

Two patterns stand out. First, cold-start ECE is identical at both scales (0.238) because without warm-up the critic predicts approximately 0.5 for every question, making calibration error invariant to model size. Second, the 3B critic is consistently less sharp than 7B across all regimes: curated ECE rises from 0.043 to 0.059, AUC drops from 0.93 to 0.85, and the top-decile success rate falls from 0.92 to 0.83. This mirrors the scale–warm-up interaction discussed in Appendix H: the larger model's broader competence surface is easier for the critic to leverage once properly initialized.

Figure 4 visualizes the underlying separation: at 7B the curated critic pushes Tier 1 and Tier 2 $V(s_0)$ distributions almost entirely apart (AUC 0.93), while at 3B the distributions overlap more (AUC 0.85) but remain clearly distinguishable.

I.1 Intermediate-State Sign Behavior

The calibration metrics in Table 11 measure V_ϕ at the start of each trajectory (s_0). CARL's segment advantages, however, are differences of V_ϕ at successive boundaries (Equation 2), so what matters operationally is whether V_ϕ also signs correctly *between* segments. This appendix tests that directly.

What we measure. For each invoke–assimilate pair, two things should happen:

- If the assimilate segment correctly extracted the answer-relevant fact, V_ϕ should **rise** from before assimilation to after (the state has become more useful).

- If retrieval surfaced the answer but the assimilate segment failed to capture it, V_ϕ should **drop** (the state has become less useful: the model has discarded what it needed).

The second case is the more interesting one. It is a within-trajectory sign divergence: the same trajectory has a positive invoke advantage (search succeeded) and a negative assimilate advantage (extraction failed). Trajectory-level methods cannot produce this pattern, because they assign a single signed scalar to every segment.

Sample and annotation. We sample 300 invoke–assimilate pairs from CARL 7B held-out trajectories (180 from HotpotQA, 120 from Musique), drawing at most one pair per trajectory so that long Musique rollouts do not dominate. A single independent annotator labels each pair on two binary axes: did retrieval surface the gold-relevant fact (i.e., is it present in the raw `stdout`), and did assimilation extract it (i.e., is it present in the `<context>` block). Both are short-span fact-presence checks against the gold answer, which makes the judgment objective enough that single-annotator labeling is sufficient.

Table 12: Sign behavior of V_ϕ at the assimilate boundary on 300 invoke–assimilate pairs (CARL 7B, held-out HotpotQA + Musique). Each row reports how often V_ϕ moved in the expected direction.

What happened in the pair	n	V_ϕ should	V_ϕ did	Rate
Retrieval succeeded, assimilation extracted the answer	175	rise	rise in 139	79.4%
Retrieval succeeded, assimilation lost the answer	86	drop	drop in 62	72.1%
Retrieval failed (out of scope here)	39	—	—	—

The first row says the critic recognises good extractions about four times in five. The second row, the diagnostic case indicates the critic also recognises information loss about seven times in ten, even though doing so requires noticing that a useful fact present a moment earlier has been discarded. The lower rate on the harder case is expected, but both rates are well above chance, and crucially both directions are observed within single trajectories: in the 62 correctly-signed diagnostic cases, the same rollout produced a positive invoke advantage and a negative assimilate advantage. This is the empirical instance of the within-trajectory sign divergence that segment-level credit makes possible and trajectory-level credit does not.

I.2 Explained Variance During PPO Training

What explained variance measures. Explained variance is the standard scalar metric for value-function quality:

$$\mathrm{EV} = 1 - \frac{\mathrm{Var}\big(R - V_\phi(s)\big)}{\mathrm{Var}(R)}$$

$\mathrm{EV} = 1$ means the critic predicts every outcome perfectly; $\mathrm{EV} = 0$ means the critic is no better than predicting the mean outcome. The metric is computed over held-out (s, R) pairs.

Why EV matters for segment-level credit. CARL's segment advantage $A(\mathrm{seg}_k) = V_\phi(s_{k+1}) - V_\phi(s_k)$ is a difference of two critic predictions. For this difference to carry a reliable sign, each prediction must be individually accurate — higher EV means each V_ϕ evaluation is closer to the true outcome, so the difference reflects genuine state-quality change rather than prediction noise. Trajectory-level GAE, by contrast, averages per-token critic errors over the full rollout, diluting individual prediction errors. Segment-level credit is therefore the more informative estimator once EV is high enough, because it preserves per-step signal that trajectory-level averaging destroys.

EV during training. Table 13 reports the explained variance of V_ϕ on a held-out evaluation set, sampled every 50 PPO steps for both scales. Step 0 is the checkpoint immediately after critic warm-up.

Table 13: Explained variance of V_ϕ on held-out trajectories, every 50 PPO steps. Bold cells mark the step at which EV first crosses 0.60 at each scale; these coincide with the steps at which CARL surpasses Search-R1 PPO in held-out EM (Figure 2(b)).

PPO step	7B EV	3B EV
0 (post warm-up)	0.52	0.48
50	0.55	0.51
100	**0.60**	0.55
150	0.65	**0.60**
200	0.70	0.64
300	0.74	0.69
400	0.77	0.73
500 (convergence)	0.79	0.76

Why the 0.60 threshold. EV $= 0.60$ is the empirically observed crossover point: below it, segment-level advantages are noisy enough that trajectory-level GAE performs comparably; above it, the per-segment signal is clean enough that CARL consistently outperforms Search-R1 PPO. The monotonic EV climb (Table 13) shows this is not a fragile threshold but a smooth transition — CARL's lead grows steadily as EV rises. The 3B critic crosses 0.60 at step $\sim$150 rather than $\sim$100 because the smaller model's critic converges more slowly; its final EV (0.76 vs 0.79) is also lower, consistent with the AUC and ECE gaps in Table 11.

J Scale Mechanisms

Why does CARL improve 3B more than 7B (1.4$\times$ ratio in main results, Table 2)? Three mechanisms compound. Table 14 reports them.

Table 14: Mechanisms across scale. All metrics computed over the 2,500 evaluation questions (500/dataset $\times$ 5 datasets).

Metric	3B	7B
Base EM without tools (mean over 5 datasets)	28%	36%
Mean $V(s_0)$ on Tier 1 questions	0.33	0.28
Mean $V(s_0)$ on Tier 2 questions	0.69	0.73
$V(s_0)$ mode separation (Tier 2 $-$ Tier 1)	0.36	0.45
Assimilation precision (init $\rightarrow$ converged)	0.42 $\rightarrow$ 0.71	0.48 $\rightarrow$ 0.79
Avg. assimilation tokens (init $\rightarrow$ converged)	215 $\rightarrow$ 118	198 $\rightarrow$ 84

(1) Larger absolute room. 3B's base EM without tools (28%) is 8 points lower than 7B's (36%), so a larger fraction of questions can in principle be moved from "wrong" to "right" by tool use. The headroom is structurally larger.

(2) Working but weaker competence boundary. 7B's $V(s_0)$ separates Tier 1 from Tier 2 with a mode-shift of 0.45 (means $0.28 \rightarrow 0.73$); 3B achieves a smaller but meaningful separation of 0.36 (means $0.33 \rightarrow 0.69$). The 3B boundary is less sharp but still functional (AUC 0.85, Table 11), so segment-level credit can still differentiate Tier 1 from Tier 2 at 3B, just less cleanly.

(3) Less compressed but still functional assimilation. Both scales learn to compress assimilation segments (token length drops from $\sim$200 to $\sim$100 over training), but 7B reaches roughly 84 tokens while 3B settles at 118, a 40% wider compression. Assimilation precision (fraction of `<context>` tokens that are answer-relevant) climbs to 0.79 at 7B versus 0.71 at 3B. The 3B critic and policy thus end with a working assimilation pipeline that uses budget less efficiently than 7B's.

The combined picture: 3B has more questions tools can fix (1), a competence boundary that lets segment-level credit do real work (2), and an assimilation step that distills retrieval into context with reasonable fidelity (3). 7B exceeds 3B on (2) and (3), but its smaller absolute room (1) caps how much CARL can change. The asymmetry produces the 1.4$\times$ scaling ratio.

K Capability Preservation and Tool-Absence Resilience

Humans learn to use tools (calculators, search engines, reference books) as a basic skill, yet retain the ability to reason without them. A useful tool-use training method should work the same way: the model should gain tool competence without losing what it already knows, and should degrade gracefully when tools are unavailable or return bad output. We test both properties.

Capability preservation. CARL's shared-backbone critic means backbone weights receive PPO gradients alongside the value head. A $10\times$ smaller learning rate on the backbone (Appendix D) limits representational drift. Table 15 confirms that CARL-trained models are indistinguishable from their base checkpoints on standard benchmarks when tools are removed at inference.

Table 15: Base Qwen2.5-Instruct vs. CARL-trained checkpoints on standard benchmarks, evaluated *without tools* at inference. Differences are within evaluation noise at both scales.

	7B			3B		
Benchmark	Base	CARL	Δ	Base	CARL	Δ
MMLU (5-shot)	74.2	74.3	$+0.1$	65.8	65.5	-0.3
GSM8K (no-tool, CoT)	91.6	91.4	-0.2	85.3	85.6	$+0.3$
TruthfulQA (MC2)	58.7	58.5	-0.2	49.1	49.3	$+0.2$
WikiText-103 (PPL $\downarrow$)	6.82	6.84	$+0.02$	8.41	8.39	-0.02

Tool-absence resilience. Preserving benchmark scores is necessary but not sufficient. The harder test is whether the model degrades gracefully when a tool is invoked but returns useless output. We force CARL to call a tool on 200 Tier 2 questions (ones it can answer parametrically) from HotpotQA and 2WikiMQA, then replace the tool output with one of four conditions: working retrieval (control), an empty string, an irrelevant passage from a different question, or malformed garbage text.

Table 16: CARL EM on Tier 2 questions when forced to invoke a tool that returns useless output. Even under the worst failure mode (malformed text), CARL retains 78.0/69.5 EM at 7B/3B. Parametric capability acts as a floor.

	7B		3B	
Tool output	EM	Δ	EM	Δ
Working retrieval (control)	85.5	—	77.5	—
Empty (no result returned)	83.0	-2.5	74.0	-3.5
Irrelevant passage (off-topic)	79.5	-6.0	71.5	-6.0
Malformed / garbage text	78.0	-7.5	69.5	-8.0

Why this works. During training, the assimilate-segment advantage $V(s_{\text{assim}+1}) - V(s_{\text{invoke}+1})$ is negative whenever retrieval does not help, teaching the model to discard unhelpful tool output rather than transcribe it. At inference, the same behavior activates: the assimilate segment compresses bad output to nothing useful, and the commit segment falls back to parametric memory, which the calibrated $V(s_0)$ had already marked as high-confidence for these Tier 2 questions (Figure 4).

The combined picture from both experiments is that CARL-trained models acquire tool competence the way humans acquire calculator skills: performance improves when the tool is available and appropriate, remains unchanged when the tool is absent, and degrades only mildly when the tool malfunctions. Teaching tool use through segment-level credit does not create tool dependence.

L Extended Related Work

This appendix provides detailed comparisons with prior work summarized in §2.

VinePPO (15). VinePPO found that token-level value networks in PPO are unreliable at ranking reasoning steps, because the value changes meaningfully only at a few tokens per trajectory while

the network must predict at every position. Our segment-level setting is structurally easier: boundaries number in single digits (typically 3–7 per episode) rather than hundreds, and each boundary corresponds to a large, observable context change (a new passage enters, an extraction replaces raw output, or a final answer is committed). The critic therefore faces a much simpler prediction problem, estimating success probability at 5 states rather than 500, with correspondingly lower variance.

PAVs (13). Process Advantage Verifiers use the same $V(s_{t+1}) - V(s_t)$ potential-based form as our segment advantages. The key difference is in how V is obtained: PAVs require either step-level annotations or a separate prover policy that can verify intermediate reasoning steps. In our setting, boundaries are organically tool-use-aligned (code fences, `</context>` tags, EOS), so no step-boundary annotation is needed, and the critic is trained from binary outcome reward on the model's own rollouts.

HICRA (5). HICRA weights the RL optimization toward high-impact "planning" tokens identified via attention analysis. While this addresses the credit-assignment problem at the token level, it operates within a single generation pass and does not model the state changes induced by tool execution. CARL operates at tool-use boundaries where state changes are observable and large, complementing rather than competing with token-level approaches.

SPO (14). Segment Policy Optimization partitions a single generation stream into segments and assigns each a Monte Carlo advantage, arriving independently at the same "segment-level credit" diagnosis as CARL. Three differences matter. (1) *Boundary definition.* SPO segments are arbitrary: fixed-length intervals or cutpoints at low-probability tokens. CARL's boundaries are environment transitions — code execution and context replacement — which is what makes the Options/SMDP framing applicable and the advantages interpretable as option values. (2) *Value estimation.* SPO is critic-free, relying on per-question Monte Carlo rollouts to estimate segment values. CARL trains a critic that generalizes across questions and encodes the model's competence boundary ($V(s_0)$); a per-question MC estimator cannot learn this cross-question structure. SPO-tree's tree-based MC sharing is a clever response to the same rollout-cost concern that motivates CARL's critic — different solutions to the same problem. (3) *Setting.* SPO is evaluated on single-turn math reasoning (no tool calls, no environment interaction). CARL targets multi-turn tool-use selectivity, where re-rollout costs make MC estimation expensive and within-trajectory sign divergence between invoke and assimilate is the diagnostic case.

ARC (3) and Agent-as-Tool (4). Both frame agentic workflow selection using Options vocabulary but train with trajectory-level GRPO rather than deriving per-option advantages from a value function. CARL takes the step these approaches stop short of: we define tool-use segments as options, derive the SMDP Bellman equation, and use the resulting TD error as the PPO advantage. The value function enables competence-aware selectivity that trajectory-level GRPO cannot produce.

EGCA (2). Execution-Grounded Credit Assignment localizes GRPO credit in code generation by comparing candidate and reference execution traces. This demonstrates that coarse credit assignment is a real bottleneck and is tractable when trajectories have structured boundaries, a finding consistent with our approach. The key difference is that EGCA requires reference solutions for execution-trace comparison, while CARL uses only binary outcome reward and structural delimiters.

RUDDER (1). Return Decomposition for Delayed Rewards redistributes episodic return across timesteps using an LSTM trained to predict the return from partial trajectories. The predicted-return differences $\hat{R}(s_{t+1}) - \hat{R}(s_t)$ have the same potential-based form as our segment advantages. RUDDER operates at the token/timestep level and requires a separate return-prediction network; our setting exploits structural boundaries to reduce the prediction problem to a handful of states per episode.

Trajectory-level RL methods: per-paper details. ToRL (7) applies trajectory-level RL to code-interpreter use and finds that intermediate execution penalties hurt performance, suggesting that naive step-level reward shaping can interfere with learning. ReSearch (21) and R3-RAG (22) extend trajectory-level training to interleaved reasoning and retrieval, showing emergent multi-step search behaviors even under a single outcome reward. These emergent behaviors confirm that LLMs can

learn multi-step tool use from trajectory-level credit, but the credit-assignment bottleneck means that good and bad tool calls within the same successful trajectory are reinforced identically. The tool-use selectivity signal is weaker still: tool calls carry intrinsic risks (irrelevant retrieval, code errors, distractors), so unnecessary tool use has a slightly elevated failure rate, but most unnecessary tool calls on easy questions still succeed and receive positive reinforcement regardless. ToolRL (9) enriches the reward with tool-name and argument-quality components. This is orthogonal to CARL: ToolRL refines what the reward evaluates; CARL changes which segment each signal reaches. A richer ToolRL-style reward could in principle be combined with CARL's segment-level credit assignment.

Search-R1/ToRL structural comparison. Table 17 summarizes the structural differences between CARL and trajectory-level methods.

Table 17: Structural comparison: Search-R1/ToRL vs. CARL.

Dimension	Search-R1 / ToRL	CARL (Ours)
RL Episode	Full trajectory	One segment per episode
Gradient masking	Required (25% perf drop without)	Not needed (by construction)
Credit assignment	Same advantage for all segments	Independent advantage per segment
Bad tool call in successful trajectory	Positive advantage	Negative advantage
Unnecessary tool call	Positive advantage (same as necessary)	Near-zero advantage (V already high)
GAE smearing	Future positive δs dilute bad signal	No smearing; clean $V(s')-V(s)$
Tool-use selectivity	No mechanism (tools always reinforced)	Explicit via segment-level $V(s_0)$
Tool output handling	Full output in context	Trainable assimilation (relevant slice)
Tool call syntax	Custom tokens	Python code blocks (no SFT)
RL algorithm	PPO or GRPO	PPO (critic essential)
Theoretical basis	Standard PPO/GRPO	SMDP + Options framework

M Hop-Count Gain Distribution

Why this analysis is needed. §4.3 claims that the 3B model benefits more than the 7B from CARL "at every hop level." Verifying this requires two things: (1) a partition of the evaluation set by hop count rather than by source dataset, and (2) a baseline against which "benefits" can be measured. Without a baseline, comparing CARL 7B to CARL 3B in absolute terms is uninformative: 7B will always be the higher number because it is a larger model. The relevant question is how much each scale *improves* relative to a controlled reference, and whether that improvement is larger at 3B than at 7B at every hop level. Search-R1 PPO shares the backbone (Qwen2.5-Inst), the training data (the 170K-prompt pool described in Appendix D), the optimizer (PPO), and the evaluation setup with CARL therefore serves as baseline.

Hop partition. Hop count is meaningful only for retrieval-based datasets, so we aggregate over HotpotQA, 2WikiMQA, and the per-hop subsets of Musique (55% 2-hop, 30% 3-hop, 15% 4-hop, per Trivedi et al. (20)). HotpotQA and 2WikiMQA are predominantly 2-hop and contribute to that bucket; 3-hop and 4-hop come from Musique only.

Table 18: Per-hop EM for CARL and Search-R1 PPO at both scales. The improvement of CARL over Search-R1 PPO (Δ columns) is larger at 3B than at 7B for every hop count, supporting the §4.3 claim.

		7B			3B		
Hop count	n	SR1-PPO	CARL	Δ	SR1-PPO	CARL	Δ
2-hop (HotpotQA + 2WikiMQA + Musique-2hop)	1,275	39.8	46.6	$+6.8$	28.2	39.4	$+11.2$
3-hop (Musique-3hop)	150	14.0	22.0	$+8.0$	7.0	16.0	$+9.0$
4-hop (Musique-4hop)	75	6.0	9.0	$+3.0$	3.0	7.0	$+4.0$

The 3B improvement exceeds the 7B improvement at every hop count: $+11.2 > +6.8$ at 2-hop, $+9.0 > +8.0$ at 3-hop, and $+4.0 > +3.0$ at 4-hop. This pattern, smaller models gaining more from CARL, is most pronounced at the easier 2-hop bucket (where 3B has the most parametric room to recover with help) and persists through to the harder 3-hop and 4-hop buckets, supporting the structural framing in the abstract.

Why both methods decline at higher hop counts. Both CARL and Search-R1 PPO show lower absolute EM as hop count increases. Each step of a multi-hop trajectory (query formulation, retrieval, extraction, reasoning to the next step) is an additional opportunity for the chain to break.

N Hallucination Analysis

We test whether CARL reduces the model's tendency to manufacture answers when its parametric memory is insufficient. Setup: 200 Tier 1 questions (sampled from HotpotQA, 2WikiMQA, Musique) where the base model fails in all 5 no-tool rollouts i.e., the model cannot answer these from parametric memory. We run four policies on this fixed sample and count how often each emits a wrong answer.

Table 19: Confident-wrong rate on 200 Tier 1 questions across four policies. CARL reduces the Search-R1 PPO confident-wrong rate by 2.7× at 7B and 2.3× at 3B; against the no-tool base it drops by 6.9× and 4.9× respectively.

Policy	7B	3B
Base Qwen2.5-Inst, no tools	62%	68%
Base Qwen2.5-Inst, selective tools	41%	50%
Search-R1 PPO	24%	32%
CARL	**9%**	**14%**

CARL learns the competence boundary and routes Tier 1 questions to tool invocation rather than guessing: confident wrong answers drop from 62% (base, no tools) to 9% at 7B, and from 68% to 14% at 3B. The 24% $\rightarrow$ 9% reduction at 7B isolates the contribution of segment-level credit specifically: Search-R1 PPO uses the same backbone, data, and tools, but its trajectory-level credit produces a model that still asserts wrong answers at nearly three times CARL's rate.

O Reproducibility Checklist

We follow standard reproducibility guidelines.

Code and data.

- All code for training, evaluation, and analysis is available at `https://anonymous.4open.science/r/carl_release-0C26`. The repository includes sample data formats for warm-up and PPO training, and converters between CARL and Search-R1 data formats.
- Training data consists of publicly available datasets: GSM8K (16) (MIT), HotpotQA (17) (CC BY-SA 4.0), 2WikiMQA (18) (Apache 2.0), FinQA (19) (MIT), and Musique (20) (CC BY 4.0).
- Base models are publicly available: Qwen2.5-7B-Instruct and Qwen2.5-3B-Instruct (Apache 2.0).
- Retrieval: BM25 over Wikipedia passages for HotpotQA, 2WikiMQA, and Musique; financial documents for FinQA; Python interpreter (no retrieval) for GSM8K. The commercial-search experiment in Appendix G uses a different retrieval source.

Experimental details.

- Full hyperparameters are reported in Appendix D.
- Warm-up dataset construction is fully described in Appendix C.
- Random seeds for all reported runs: $\{1, 2, 3\}$.
- Evaluation uses deterministic decoding (greedy) with temperature 0.0 for final numbers; training rollouts use temperature 1.0 with top-p 1.0.

Compute.

- Training cluster: Azure Machine Learning (AML) with 4 nodes of 8× NVIDIA H100 80GB each (32 GPUs total). Each training run uses 8 GPUs (one node); multiple runs were executed in parallel.

- Evaluation, inference, and tier-label construction were run on AMD MI300X GPUs; all figures below are translated to single-H100-equivalent hours for comparability.
- Total compute for all reported numbers: approximately 5,500 H100-hours. Total project compute including supporting work: approximately 22,000 H100-hours (Table 5).
- Training time per run: $\sim$50 hours at 7B, $\sim$22 hours at 3B.

Statistical reporting.

- Main results for CARL and Search-R1 PPO report mean $\pm$ std over 3 seeds $\{1, 2, 3\}$. Prompting baselines and Search-R1 GRPO are single runs.
- Ablation and analysis seed conventions:
  - Appendix H (warm-up variants): single run per condition; cold and random use seed 1.
  - Appendix G (retrieval ablation): eval-only on fixed CARL seed-1 checkpoint.
  - Appendix K, tool-absence resilience experiment (Table 16): eval-only on fixed CARL seed-1 checkpoint.
  - Appendix I (calibration data): aggregated across all 3 seeds for CARL.

Computational framework.

- CARL is implemented in OpenRLHF (24). Search-R1 PPO baselines are reproduced in verl following the original Search-R1 codebase (6) but using our training split; the critic head is warmed up identically to CARL's. Our Search-R1 PPO reproduction achieves higher EM than the numbers reported in the original paper, confirming a fair-or-favorable baseline.

Human annotations.

- Appendix F (Musique error categorization) and Appendix N (hallucination analysis) used 2 independent annotators.